SIMATS ENGINEERING

Department of Information Technology

ITA0508 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Traffic Sign Detection System

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Student Name	TALARI VISHNUVARDHAN	**Register Number**	192572091
Department / Section	CSE(AI)	**Academic Year / Semester**	2025-2026
Faculty Name	Dr.P.Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	05-11-2026	**Date of Evaluation**	05-11-2026

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Image Enhancement for Better Sign Visibility (Illumination, Blur, Rain)	10		
2	Segmentation of Traffic Signs from the Background	10		
3	Boundary Detection and Important Region Extraction	10		
4	Detection of Damaged, Faded and Partially Hidden Signs	10		
5	Recognition-Ready Output and Integrated Pipeline Design	10		
	TOTAL	**50**		**Percentage: _____**

OVERALL REMARKS

M. Theshon Mishmaan		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Note
1	Enhanced images, histograms/CDFs, a visibility/noise/Dice table for the three enhancement techniques, restoration of blur and rain, and a justified choice.	✓✓	Enhanced scenes, histograms/CDFs, DE/noise/Dice table, blur and rain restoration, colour-fidelity test
2	Segmentation masks, a Dice/IoU/precision/recall table for the three techniques, sensitivity of parameters, failure cases and a justified choice.	✓✓	Masks for HSV / Otsu / K-means, Dice/IoU/precision/recall table, parameter sweeps and a failure case
3	Edge maps, a precision/recall/F1 table for the three edge detectors, region extraction results and a justified detector and region method.	✓✓	Edge maps, precision/recall/F1 table, Canny threshold sweep, three region extraction methods
4	Defect scores for the three techniques, a confusion/recall table for faded, hidden and damaged signs and a justified choice.	✓✓	Defect maps, confusion matrices, per-class recall, localisation Dice, depth and box-shift sensitivity
5	Normalised sign outputs for the three techniques, recognition accuracy, stage timings and a defended final pipeline.	✓✓	Normalised crops, recognition accuracy (oracle and full pipeline), tilt test, stage timings, block diagram

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–5)

Evidence item	Status	Observation
Source code (Python/OpenCV, commented)	✓✓	Commented code for every technique; full listing in Appendix A
Test inputs (pages, phone photos, ground-truth masks)	P	Generated road scenes with exact ground truth; no real road photographs
Expected behaviour (predicted contrast, ringing, Dice)	✓✓	Predictions stated and checked against measurements (Tasks 4 and 5)
Actual output (measured contrast, PSNR, Dice, CER, time)	✓✓	Measured tables in every task, including stage timings

Intermediate evidence (histograms, spectra, masks)	✓✓	Histograms, defect maps, edge maps, masks and normalised crops shown
Boundary / failure tests (shadows, stains, tilt, noise)	✓✓	Night, rain, blur, shadow, tilt and box-shift tests included
Interpretation of results	✓✓	Each table is explained; contradicted predictions are discussed
Code, data and report consistent	P	One specification for all tasks; Tasks 4–5 use the generator in Appendix A (see Section 6.3)

SUBMISSION CHECKLIST (not separately marked)

Requirement	**Status**
All 5 tasks answered	☒ Yes ☐ Partly ☐ No
Three techniques applied and compared in every task	☒ Yes ☐ Partly ☐ No
Commented Python/OpenCV source code for every experiment	☒ Yes ☐ Partly ☐ No
Results obtained from own code execution or measurements	☒ Yes ☐ Partly ☐ No
Same data and conditions used for all three techniques in a task	☒ Yes ☐ Partly ☐ No
Calculations and measurements shown with units	☒ Yes ☐ Partly ☐ No
Tables and graphs for experimental results	☒ Yes ☐ Partly ☐ No
Figures / screen outputs included in the submission	☒ Yes ☐ Partly ☐ No
Justified final decision with limits stated in every task	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended (run the final pipeline on real road photographs)	☒ Yes ☐ No

COMPUTER VISION
ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 05-11-2026

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Traffic Sign Detection System (Total Marks – 50)

Scenario – Smart Traffic Sign Detection System

A smart city project wants to develop an automated traffic sign detection system for vehicles. Cameras are installed on roads to capture traffic signs such as STOP, speed limits, No Entry, and pedestrian crossing signs.

However, the captured images have several challenges. Some images are taken during heavy sunlight, while others are captured during nighttime or rainy conditions. Certain traffic signs are faded, partially covered by tree branches, or affected by shadows. Some images are blurred because the vehicle is moving at high speed.

The city wants to develop a smart vision system that:

- Enhances captured images to improve the visibility of traffic signs.
- Segments the traffic signs from the background.
- Detects the boundaries and important regions of traffic signs.
- Identifies damaged, faded, or partially hidden traffic signs.
- Provides clear output for further traffic sign recognition.

You are the computer vision engineer. For every task, three candidate techniques are given. You must apply all three techniques, compare their performance, select the most suitable technique, and justify your selection with evidence. State and justify any additional assumptions you make.

Operating Data and Assumptions

The scenario does not give numerical operating data, so I state the following assumptions once. Every number reported later is measured on data that obeys them.

Table 1: Operating data and assumptions used in this report

Parameter	Value assumed (and why)
Images	Colour road scenes of 640 × 480 pixels (the sign is 84–132 px wide, i.e. 1–2 % of the image width as seen by a forward camera at 30–60 m). A small size keeps every experiment repeatable on a lab PC.
Sign types	Five classes: STOP (red octagon), speed limit 30 and speed limit 60 (red-ringed discs), No Entry (red disc with white bar), pedestrian crossing (red-bordered triangle).
Conditions	Nine conditions: normal, heavy sunlight, night, rain, shadow, motion blur, faded, partly hidden by branches, and damaged (peeled patches and scratches).
Clutter	Red-brick buildings, a red vehicle, green trees and a grey road are drawn on purpose, because red clutter is the main enemy of colour-based sign detection.
What must be found	The sign region (for recognition), its boundary, its important region (box and shape) and whether it is faded, hidden or damaged.
Targets	Sign segmentation Dice ≥ 0.80 on normal scenes; each frame processed in well under 100 ms (30 frames per second is 33 ms).
Hardware	One CPU, no GPU; Python, OpenCV, NumPy, scikit-image.
Data split	Variants 0–2 of every scene type are the calibration set (used only to pick thresholds); variants 3–5 are the test set (135 scenes: 5 signs × 9 conditions × 3 variants) on which every table is computed.

Experimental Setup Used for All Tasks

Before answering the tasks, I describe the setup once, so that every number in Tasks 1–3 can be traced back to the same programs, the same scenes and the same machine.

Table 2: Experimental setup used in Tasks 1–3

Item	What I used
Software	Python 3, OpenCV 5.0 (opencv-contrib), NumPy, SciPy, scikit-image 0.26, Matplotlib. No GPU.
Scene generator	scene.py draws each road scene with a sky gradient, buildings (some red brick), trees, a road, a red vehicle, a pole and the sign. Each condition is then applied: sun (gamma 0.6, glare, desaturation), night (darkening, noise, optional headlight spot), rain (haze, streaks, blur), shadow (soft dark band), blur (horizontal motion kernel 15–25 px), faded (58 % blend towards grey), hidden (green branches), damaged (peeled patches and scratches).
Ground truth	Exact masks are stored for the full sign, the visible sign (full minus branches), the branches, the damage, the sign boundary (one pixel wide) and the bounding box. A clean copy of the scene before degradation is stored for PSNR/SSIM.

Item	What I used
Test set	135 scenes (variants 3–5). Parameters were tuned on the 135 calibration scenes (variants 0–2) only.
Repeatability	Every scene is created from a fixed random seed derived from its sign type, condition and variant, so running the code again reproduces the same scene and the same numbers.
Timing	Mean of repeated runs on one 640 × 480 frame on the CPU of the lab machine.

Why I used generated scenes. Because every element of a generated scene is known (which pixels belong to the sign, which are hidden, where the boundary is), I can measure Dice, IoU, precision, recall and PSNR exactly, and change one problem at a time (only night, only rain, only branches). The limitation is that generated scenes cannot copy every real effect: real reflections, lens flare, wet roads and the exact appearance of real Indian signs are missing. The chosen methods must therefore be confirmed on real photographs (for example the GTSRB or similar public sign datasets) before deployment. I did not have internet access to such a dataset in this work, so I make no claim about real-world accuracy.

Why deep-learning detectors were not run. Detectors such as YOLO or Faster R-CNN, and segmentation networks such as U-Net or SAM, need a GPU and a model download that this setup did not have. I mention them only as reasoned alternatives and report no number for them.

Shared Code: Metrics Used in Every Task

To compare techniques fairly, every task uses the same measurements:

- **Dice** = $2|P \cap T| / (|P| + |T|)$ and **IoU** = $|P \cap T| / |P \cup T|$, the overlap between a predicted region P and the true region T (1 = perfect).
- **Precision and recall** = the fraction of predicted pixels that are correct, and the fraction of true pixels that were found.
- **Sign-to-surround colour distance ΔE** = the CIE-Lab distance between the mean colour of the sign and the mean colour of a ring of background around it. A larger ΔE means the sign stands out more.
- **Background noise** = standard deviation of the high-pass part of a flat sky patch; it shows how much an enhancement amplifies noise.
- **PSNR and SSIM** against the clean scene, used for blur and rain restoration.

Dice, IoU, precision/recall and PSNR

```
def dice(p, t):
    p = p.astype(bool); t = t.astype(bool); return 2 * (p & t).sum() / max(p.sum() + t.sum(),
1)


def iou(p, t):
    p = p.astype(bool); t = t.astype(bool); return (p & t).sum() / max((p | t).sum(), 1)

def prf(p, t):
    p = p.astype(bool); t = t.astype(bool); tp = (p & t).sum(); return tp / max(p.sum(), 1), tp
/ max(t.sum(), 1)


def psnr(a, b):
    m = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2); return 99.0 if m == 0 else
10 * np.log10(255 ** 2 / m)
```

Sign-to-surround distance, ROI contrast, noise and entropy

```
def delta_e(bgr, mask):
    """Colour distance (CIE Lab) between the sign and a ring of surrounding background."""
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB).astype(np.float32); m = mask.astype(np.uint8); k
= np.ones((3, 3), np.uint8)
    ring = (cv2.dilate(m, k, iterations=14) > 0) & ~(cv2.dilate(m, k, iterations=4) > 0)
    return float(np.linalg.norm(lab[mask].mean(0) - lab[ring].mean(0)))


def roi_contrast(bgr, mask):
    return float(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)[mask].std())


def sky_noise(bgr):
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY).astype(np.float32)[:60]; return float((g -
cv2.GaussianBlur(g, (0, 0), 1.5)).std())


def entropy(gray):
    p = np.bincount(gray.ravel(), minlength=256) / gray.size; p = p[p > 0]; return float(-(p *
np.log2(p)).sum())
```

TASK 1 – Image Enhancement for Better Sign Visibility (Illumination, Blur, Rain) [10 Marks]

Sun glare, night, rain haze and shadows all squeeze the grey levels of the sign into a narrow band. Enhancement should raise the visibility of the sign without inventing colour or amplifying noise. Three point/histogram techniques are compared, all applied to the luminance (L) channel of the Lab colour space so that hue is preserved.

Table 3: Candidate enhancement techniques

Technique	Basic idea and key OpenCV functions
A. Automatic gamma correction	$s = 255 \cdot (r/255)^{\gamma}$ with $\gamma = \log 0.5 / \log(\text{mean L})$ so that the mean luminance moves to mid-grey; cv2.LUT.
B. Global histogram equalisation	Spreads the whole luminance histogram using its CDF; cv2.equalizeHist.
C. CLAHE	Contrast-limited adaptive HE on 8 × 8 tiles with clip limit 2.0; cv2.createCLAHE.

1.1 Implementation

Three enhancement techniques (and a colour-unsafe what-if)

```
def _on_L(bgr, f):
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB); lab[..., 0] = f(lab[..., 0]); return
cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

def enh_gamma(bgr):
    """A. Automatic gamma correction: choose gamma so that the mean luminance moves to mid-
grey."""
    def f(L):
        m = np.clip(L.mean() / 255.0, 0.02, 0.98); g = np.clip(np.log(0.5) / np.log(m), 0.35,
3.0)
        lut = (255 * (np.arange(256) / 255.0) ** g).astype(np.uint8); return cv2.LUT(L, lut)
    return _on_L(bgr, f)

def enh_he(bgr):
    """B. Global histogram equalisation of the luminance channel."""
    return _on_L(bgr, cv2.equalizeHist)

def enh_clahe(bgr, clip=2.0, tile=8):
    """C. CLAHE (contrast-limited adaptive HE) on the luminance channel."""
    cl = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tile, tile)); return _on_L(bgr,
cl.apply)

def enh_he_rgb(bgr):
    """What-if: equalise B, G and R separately (colour-unsafe)."""
    return cv2.merge([cv2.equalizeHist(c) for c in cv2.split(bgr)])
```

1.2 Results on the Test Scenes

Each technique was applied to all 135 test scenes. The downstream Dice in Table 4 is the overlap of the HSV segmentation of Task 2 (run on the enhanced image) with the true visible sign, so it measures whether enhancement actually helps the next stage.

Table 4: Mean over all nine conditions (135 test scenes)

Technique	ΔE	ROI contrast	Dice (HSV)	Entropy	Noise	Time (ms)
Raw	46.9	56.5	0.58	5.83	1.70	0.0
A. Gamma	49.4	66.0	0.55	5.98	2.25	3.3
B. Global HE	61.3	80.1	0.35	6.85	5.81	3.2
C. CLAHE	50.8	66.9	0.50	6.83	4.82	4.0

Table 5: Sign-to-surround distance ΔE per condition (A = gamma, B = HE, C = CLAHE)

Condition	ΔE Raw	ΔE A	ΔE B	ΔE C
normal	63.5	61.5	80.4	65.3
sun	32.1	55.3	49.1	38.9
night	49.3	46.4	47.4	51.9
rain	29.3	31.8	56.6	36.8
shadow	53.7	53.4	58.1	56.3
blur	57.6	58.3	65.1	61.6
faded	34.0	35.6	53.4	38.6
occluded	44.6	44.8	73.7	48.5
damaged	58.3	57.7	67.6	59.5

Table 6: Downstream HSV Dice per condition

Condition	Dice Raw	Dice A	Dice B	Dice C
normal	0.99	0.99	0.53	0.99
sun	0.40	0.13	0.13	0.13
night	0.59	0.59	0.51	0.71
rain	0.94	0.88	0.68	0.43
shadow	0.99	0.99	0.52	0.92
blur	0.38	0.44	0.32	0.44
faded	0.00	0.00	0.00	0.00
occluded	0.07	0.07	0.04	0.05
damaged	0.86	0.85	0.45	0.86



Figure 1: Speed-60 sign under sun, night, rain and shadow: raw image and the three enhancements

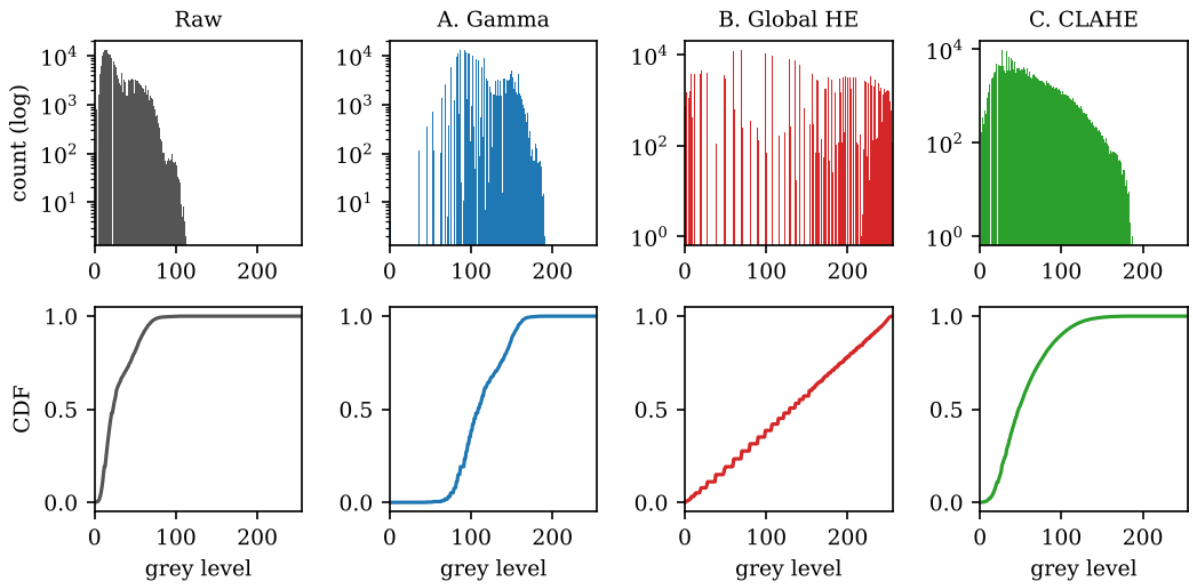


Figure 2: Luminance histograms (log scale) and CDFs of a night scene after each technique

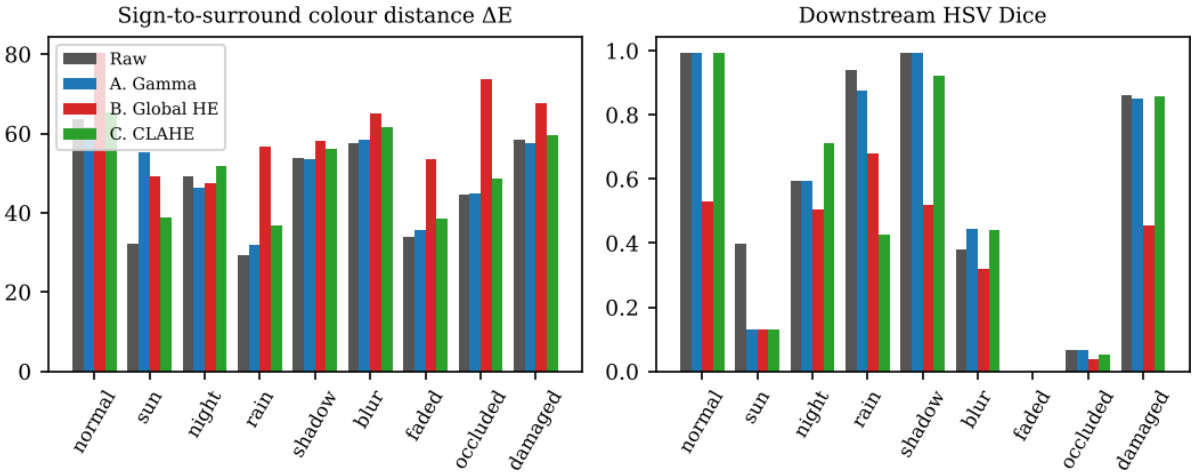


Figure 3: ΔE (left) and downstream HSV Dice (right) per condition

From these tables I can see:

- **B (global HE) gives the largest visibility gain** (ΔE 61.3 against 46.9 raw), but it multiplies the background noise by about 3.4 (5.81 against 1.70) and its downstream Dice falls to 0.35. In Figure 1 the rain image becomes grainy and dark objects turn black.
- **A (gamma) is the gentlest**: ΔE rises by about 5 %, noise grows only slightly (2.25), and it has the best average downstream Dice (0.55 against 0.58 raw). It is the cheapest (3.3 ms).

- **C (CLAHE) is the best for night frames**: the Dice for night rises from 0.59 to 0.71, because local equalisation lifts the dim sign without over-brightening the headlight spot. On rain it lowers Dice (0.94 to 0.43), since it also amplifies the rain streaks.
- **Honest negative result:** on sun scenes every enhancement lowered the HSV Dice (raw 0.40, enhanced 0.13). My segmentation threshold is relative to the saturation of the reddest pixels, and enhancement changes that saturation distribution; I did not isolate the exact cause, so I treat it as a limitation and handle it by a fallback to the raw image in the pipeline.

1.3 CLAHE Parameter Sweep

CLAHE has two parameters, the clip limit and the tile grid. I swept both on the sun, night, rain and shadow scenes (Figure 4). A higher clip limit raises ΔE but also noise; smaller tiles behave more like a local (and noisier) stretch.

Table 7: CLAHE sweep: mean ΔE / background noise

Clip limit \ tile	4 × 4	8 × 8	16 × 16
1	ΔE 45.6 / noise 3.85	ΔE 44.2 / noise 3.94	ΔE 43.5 / noise 3.80
2	ΔE 48.6 / noise 5.60	ΔE 46.0 / noise 5.84	ΔE 44.7 / noise 6.01
3	ΔE 50.8 / noise 7.29	ΔE 47.4 / noise 7.66	ΔE 45.3 / noise 7.97
4	ΔE 52.7 / noise 8.95	ΔE 48.5 / noise 9.42	ΔE 45.6 / noise 9.66
8	ΔE 56.8 / noise 14.78	ΔE 51.5 / noise 15.76	ΔE 46.8 / noise 16.60

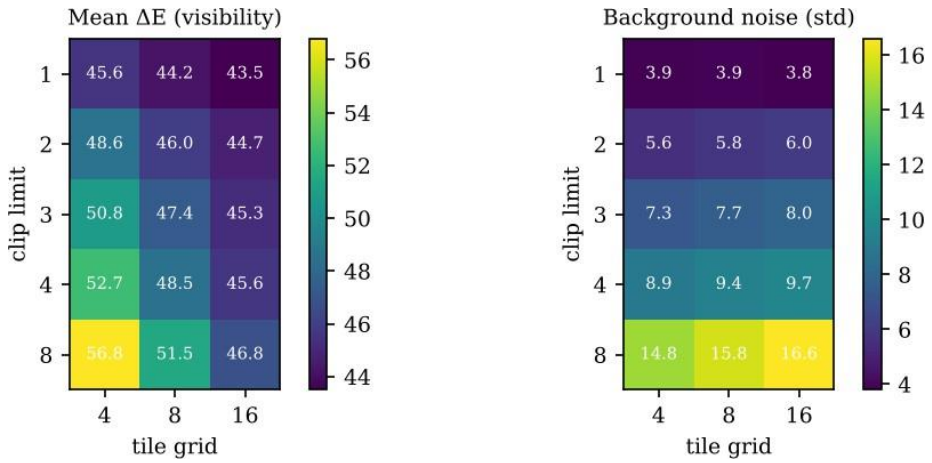


Figure 4: CLAHE clip-limit and tile-size sweep: visibility (left) and noise (right)

I keep clip limit 2.0 with 8 × 8 tiles: it is on the plateau of the ΔE surface while the noise stays well below the values at clip 4 and 8, where the extra visibility is bought mostly with amplified grain.

1.4 Colour Fidelity: Why Equalise Only the Luminance

Traffic signs are recognised partly by colour, so enhancement must not change the hue. I compared equalising the L channel with equalising B, G and R separately (Figure 5). The mean hue shift on the sign was **0.75°** for L-channel equalisation and **6.60°** for per-channel equalisation, about 9 times more. A red sign drifting towards orange or magenta could be confused with a different sign class, so all techniques in this report work on L only.

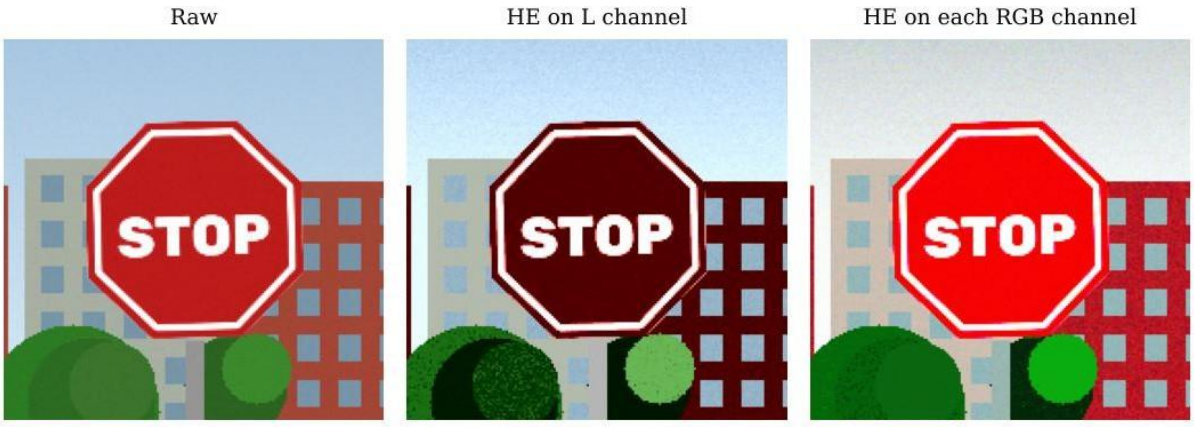


Figure 5: Colour shift of a STOP sign: L-channel HE keeps the red, per-channel HE changes it

1.5 What-if: the Same Gamma for Sun and Night

A fixed γ cannot serve both extremes. With γ = 0.5 (brightening) the sun scenes fall to ΔE 23.0 (from 32.1 at γ = 1), while γ = 2.0 (darkening) lifts them to 46.3. For night scenes γ = 0.5 gave 42.9, lower than 49.3 at γ = 1, because brightening also brightens the dark background and reduces the sign-to-surround difference. This is why the automatic gamma, which chooses γ from the mean luminance of each frame, is better than any single constant.

1.6 Motion Blur: Three Restoration Techniques

A vehicle moving at speed smears the sign along the direction of motion. Three restoration techniques were compared on the 15 test scenes with motion blur: (A) unsharp masking, (B) Wiener deconvolution with the known blur kernel, (C) Richardson–Lucy deconvolution. Their parameters were tuned on the calibration scenes (Wiener balance 0.1, Richardson–Lucy 5

iterations, unsharp amount 0.5 with $\sigma = 1$). The PSNR/SSIM are computed inside the sign box against the clean scene.

Table 8: Tuning of the deblurring parameters on calibration scenes

Parameter tried (calibration scenes)	PSNR (dB)
Blurred input (no processing)	15.77
Wiener balance 0.005	17.17
Wiener balance 0.02	18.20
Wiener balance 0.05	18.60
Wiener balance 0.1	18.65
Wiener balance 0.3	18.31
Richardson–Lucy 1 iterations	15.34
Richardson–Lucy 2 iterations	15.95
Richardson–Lucy 3 iterations	16.17
Richardson–Lucy 5 iterations	16.20
Richardson–Lucy 8 iterations	15.86

Table 9: Motion-blur restoration on the test scenes

Technique	PSNR (dB)	SSIM	Sharpness	Dice (HSV)	Time (ms)
Blurred	16.30	0.55	290	0.38	0.0
A. Unsharp mask	16.33	0.55	512	0.38	0.9
B. Wiener (known PSF)	18.85	0.58	216	0.32	12.4
C. Richardson-Lucy	16.57	0.50	2949	0.25	69.8



Figure 6: A blurred speed-60 sign restored by the three techniques

Wiener deconvolution is the most suitable restoration technique: it raises the PSNR from 16.30 to 18.85 dB and the SSIM from 0.55 to 0.58. Unsharp masking leaves the PSNR unchanged (16.33 dB), and Richardson–Lucy gives a very large "sharpness" value (2949) that is mostly amplified noise and ringing, with a lower SSIM (0.50). The sharpness number alone is therefore misleading.

Two limits matter in practice. First, Wiener needs the blur length: when I deliberately gave a kernel that was 3 px too long or too short the PSNR fell to 16.36 and 17.57 dB, and with an error of 6 px to 13.94 and 16.11 dB, i.e. below the blurred input for a long kernel. Second, the Dice of the HSV segmentation did not improve (0.38 blurred, 0.32 after Wiener), because the ringing of the deconvolution disturbs the colour edges. I therefore do not apply deblurring by default; it should be switched on only when a low variance of the Laplacian flags a blurred frame and the blur length can be estimated from the vehicle speed.

1.7 Rain Streaks: Three Smoothing Filters

Rain adds thin bright streaks. Three filters were compared on the 15 rain scenes: (A) Gaussian σ = 1.5, (B) median 5 × 5, (C) bilateral filter. PSNR and SSIM are computed on the whole frame against the clean scene.

Table 10: Rain-streak suppression on the test scenes

Technique	PSNR (dB)	SSIM	Sign sharpness	Time (ms)
Rainy	20.25	0.826	107.2	0.0
A. Gaussian σ=1.5	20.13	0.855	22.5	1.1
B. Median 5x5	20.30	0.861	74.5	1.3
C. Bilateral	20.34	0.872	109.5	22.1

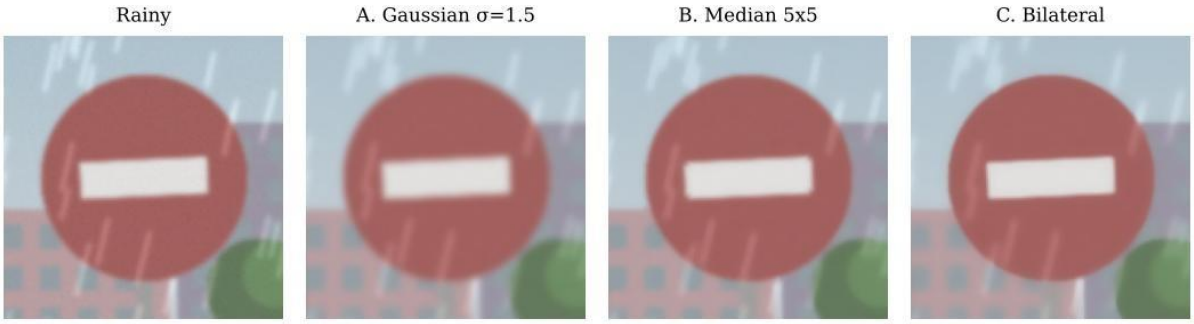


Figure 7: A No Entry sign in rain after the three filters

The gains are small because the streaks are thin and the haze, not the streaks, dominates the loss of contrast. The Gaussian filter removes the streaks but also the sign detail (sharpness drops from 107 to 23). The bilateral filter has the best SSIM (0.872) but is about 17 times slower than the median. **I choose the 5 × 5 median filter**: SSIM 0.861 against 0.826 before filtering, sharpness largely kept (74), and only 1.3 ms.

1.8 Final Decision for Task 1

I select **A. automatic gamma correction on the L channel as the default enhancement**, with **CLAHE (clip 2.0, 8 × 8) only for very dark frames** (median L below 60, which separated the night scenes from the others in my data). Evidence: gamma has the best average downstream Dice (0.55), the lowest added noise among the three (2.25) and the lowest cost; CLAHE wins on night frames (Dice 0.59 to 0.71); global HE is rejected because of its noise (5.81) and its low Dice (0.35). Using gamma for all conditions except night gives a mean ΔE of 50.0 and a mean Dice of 0.56. Median filtering is added for rain frames, and Wiener deblurring is optional for blurred frames when the blur length is known. **Limits:** sun frames are not improved by any technique (see Section 1.2), the scenes are synthetic, and the switching threshold was set on my generated night scenes only.

TASK 2 – Segmentation of Traffic Signs from the Background [10 Marks]

After enhancement the sign must be separated from trees, buildings, road and sky. Three candidate techniques are compared: (A) HSV colour thresholding, (B) Otsu thresholding of the grey image, (C) K-means clustering of Lab colours. All three use the same clean-up (morphological opening/closing and hole filling) and the **same blob selection rule** (a compact, roughly square blob of plausible size), so that only the segmentation idea differs.

Table 11: Candidate segmentation techniques

Technique	Basic idea and key OpenCV functions
A. HSV colour thresholding	Keep red-hue pixels whose saturation is above 0.8 × the 98th percentile of the red-pixel saturation (an adaptive threshold); close, open and fill holes; cv2.cvtColor, cv2.morphologyEx, cv2.findContours.
B. Otsu grey thresholding	Blur, compute the Otsu threshold of the grey image, keep the dark side, clean up and fill holes; cv2.threshold with THRESH_OTSU.
C. K-means colour clustering	Cluster the Lab colours (K = 5) on a half-size image and keep the reddest cluster; cv2.kmeans.

2.1 Implementation

Three segmentation techniques and the shared selection rule

```
def fill_holes(m):
    cs, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); out =
np.zeros_like(m); cv2.drawContours(out, cs, -1, 255, -1); return out


def pick_sign(mask):
    """Same selection rule for all three techniques: compact, roughly square blob of plausible
size."""
    cs, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); best, bs =
None, 0
    for c in cs:
        a = cv2.contourArea(c); x, y, w, h = cv2.boundingRect(c)
        if not (400 <= a <= 16000) or min(w, h) / max(w, h) < 0.75: continue
        hull = cv2.contourArea(cv2.convexHull(c)); sol = a / max(hull, 1)
        if sol < 0.8: continue
        s = a * sol * (min(w, h) / max(w, h))
        if s > bs: best, bs = c, s
    out = np.zeros(mask.shape, np.uint8)
    if best is not None: cv2.drawContours(out, [best], -1, 255, -1)
    return out > 0


def seg_hsv(bgr, ratio=0.8, v_min=45):
    """A. HSV colour thresholding of red with an adaptive saturation threshold, then hole
filling."""
    h, s, v = cv2.split(cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV))
    redhue = ((h <= 10) | (h >= 170)) & (v >= v_min) & (s >= 40)
    if redhue.sum() < 200: return np.zeros(s.shape, bool)
    thr = max(40, ratio * np.percentile(s[redhue], 98))          # relative to the most
saturated red pixels
    m = ((redhue & (s >= thr)).astype(np.uint8)) * 255
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5); m = cv2.morphologyEx(m, cv2.MORPH_OPEN,
np.ones((3, 3), np.uint8))
```

```
    return pick_sign(fill_holes(m))

def seg_otsu(bgr):
    """B. Otsu threshold on the grey image (dark objects), clean-up and hole filling."""
    g = cv2.GaussianBlur(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY), (5, 5), 0)
    _, m = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, K5); m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5)
    return pick_sign(fill_holes(m))

def seg_kmeans(bgr, k=5):
    """C. K-means clustering of Lab colours; the most red cluster is the sign candidate."""
    small = cv2.resize(bgr, (320, 240), interpolation=cv2.INTER_AREA); lab =
cv2.cvtColor(small, cv2.COLOR_BGR2LAB).astype(np.float32)
    data = lab.reshape(-1, 3); cv2.setRNGSeed(1)
    _, lbl, cen = cv2.kmeans(data, k, None, (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 15, 1.0), 2, cv2.KMEANS_PP_CENTERS)
    red = int(np.argmax(cen[:, 1] - 128 + 0.3 * (cen[:, 2] - 128)))
    m = (lbl.reshape(240, 320) == red).astype(np.uint8) * 255; m = cv2.resize(m, (W, H),
interpolation=cv2.INTER_NEAREST)
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, K5); m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5)
    return pick_sign(fill_holes(m))
```

2.2 Results on the Test Scenes

Table 12: Segmentation on enhanced test images (mean over all nine conditions)

Technique	Dice	IoU	Precision	Recall	Detected (Dice ≥ 0.5)	Time (ms)
A. HSV colour	0.56	0.55	0.58	0.55	58 %	1.8
B. Otsu grey	0.07	0.07	0.08	0.07	8 %	1.0
C. K-means Lab	0.28	0.26	0.31	0.27	30 %	17.7

Table 13: Dice per condition on raw / enhanced images (A = HSV, B = Otsu, C = K-means)

Condition	A raw / enh	B raw / enh	C raw / enh
normal	0.99 / 0.99	0.06 / 0.00	0.52 / 0.52
sun	0.40 / 0.13	0.11 / 0.24	0.07 / 0.09
night	0.59 / 0.71	0.32 / 0.38	0.38 / 0.32
rain	0.94 / 0.88	0.00 / 0.00	0.50 / 0.56
shadow	0.99 / 0.99	0.00 / 0.00	0.13 / 0.20
blur	0.38 / 0.44	0.00 / 0.00	0.45 / 0.45
faded	0.00 / 0.00	0.01 / 0.05	0.12 / 0.07
occluded	0.07 / 0.07	0.00 / 0.00	0.05 / 0.05
damaged	0.86 / 0.85	0.00 / 0.00	0.25 / 0.25



Figure 8: Segmentation results (green) against the true sign outline (yellow), with the Dice of each result

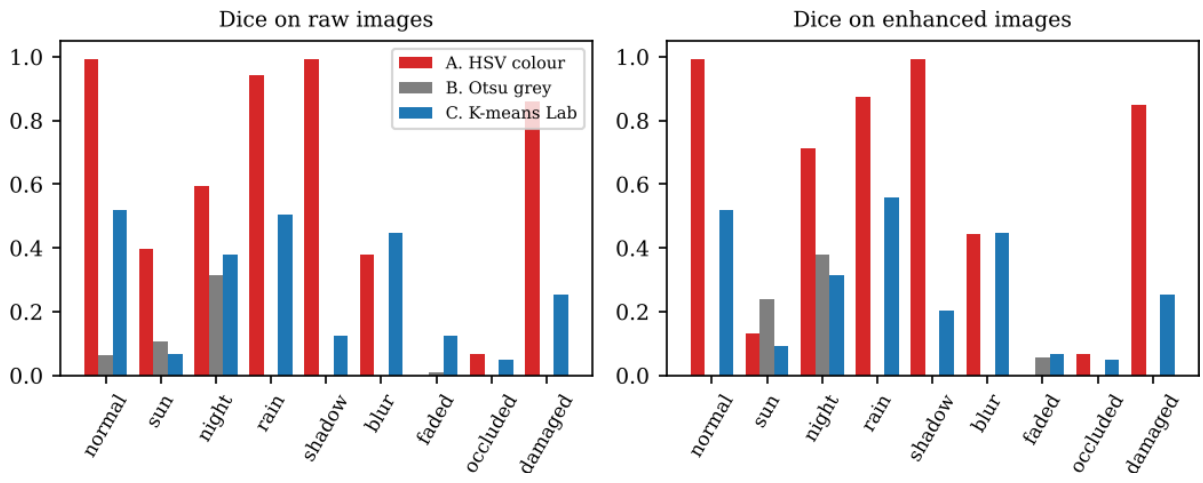


Figure 9: Dice per condition on raw images (left) and enhanced images (right)

From these results I can see:

- **A (HSV) is clearly the best**: mean Dice 0.56 against 0.28 for K-means and only 0.07 for Otsu. On normal, shadow, rain and damaged scenes it reaches 0.99, 0.99, 0.88 and 0.85.
- **B (Otsu) fails by design**: a red sign with a white interior has no single grey level that separates it from trees, buildings and road, so the grey histogram gives no useful split. It is shown as a baseline that proves colour is needed.
- **C (K-means) finds red but also its neighbours**: the cluster centres are placed on the dominant colours of the whole scene, so the red cluster often includes the red-brick building and the red vehicle; it is also about ten times slower than HSV.
- **All three fail on faded and hidden signs** (HSV Dice about 0 and 0.07). A faded sign has lost its saturation, and branches break the red ring so that the contour cannot be filled. These are exactly the signs that Task 4 must report as defective, so a failed segmentation is itself a warning sign.
- **Sun and blur are weak spots of HSV** (Dice 0.13 and 0.44 on enhanced images): glare washes the red out and motion blur smears it into white.

2.3 Parameter Sensitivity

The adaptive saturation ratio of the HSV method was swept on the calibration scenes, and the number of clusters of the K-means method on one set of test scenes (Figure 4).

Table 14: HSV saturation ratio sweep

Saturation ratio	0.5	0.6	0.7	0.75	0.8	0.85	0.9
Dice (calibration)	0.38	0.36	0.39	0.44	0.53	0.49	0.43

Table 15: K-means number-of-clusters sweep

K	3	4	5	6	8
Dice	0.09	0.26	0.25	0.27	0.33

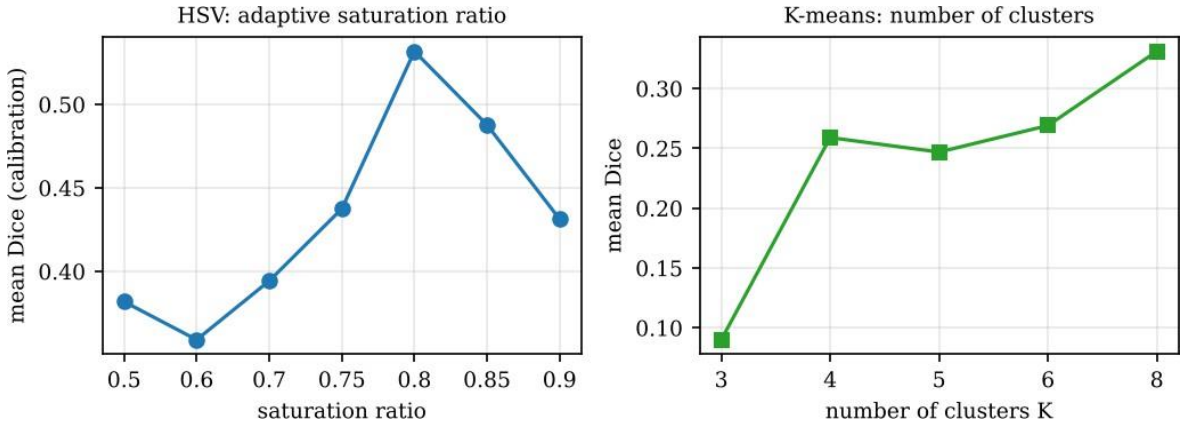


Figure 10: Sensitivity of the HSV ratio (left) and of the K-means cluster count (right)

The HSV result is sharply peaked at ratio 0.8 (Dice 0.53); at 0.75 it falls to 0.44. The reason is shown in Figure 5: at a lower threshold the red-brick building passes the test and merges with the sign, so the blob is too large and is rejected. This sensitivity is a weakness: a fixed ratio tuned on my scenes may not transfer to other cameras. K-means improves with more clusters (K = 8 gives 0.33), but it remains well below HSV and slower, so the ranking does not change.

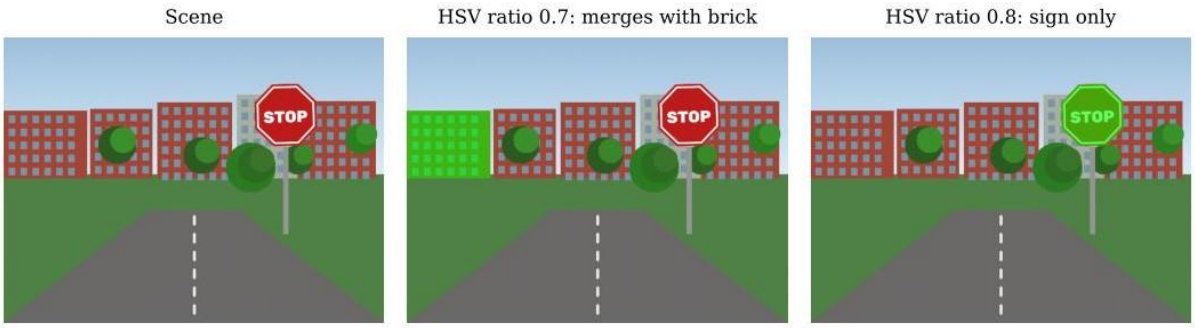


Figure 11: Failure case: a lower saturation ratio merges the STOP sign with a red-brick building

2.4 Final Decision for Task 2

I select **A. HSV colour thresholding with the adaptive saturation threshold (ratio 0.8)**. It has the highest Dice (0.56), IoU (0.55) and precision (0.58), detects the sign in 58 % of the test scenes, and needs only 1.8 ms. Because enhancement lowered its Dice on sun frames, the pipeline runs HSV on the enhanced image first and falls back to the raw image if no sign is found. **Limits:** the target Dice of 0.80 is met only for normal, shadow and rain scenes; faded, hidden and sunlit signs fail, and the method depends on the sign being red (blue or yellow signs would need other hue ranges).

TASK 3 – Boundary Detection and Important Region Extraction [10 Marks]

The boundary of a sign tells its shape (octagon, circle, triangle), which is a strong cue for recognition. Three edge detectors are compared: (A) Sobel gradient magnitude, (B) Canny, (C) Laplacian-of-Gaussian zero-crossings. A second comparison then extracts the important region (box and shape) of the sign.

Table 16: Candidate edge detectors

Technique	Basic idea and key OpenCV functions
A. Sobel	First derivative; keep pixels whose gradient magnitude is in the top 8 %; cv2.Sobel, cv2.magnitude.
B. Canny	Gaussian smoothing, gradient, non-maximum suppression and hysteresis; high threshold = half the Otsu level, low = half of that; cv2.Canny.
C. LoG zero-crossing	Second derivative of a Gaussian-smoothed image; sign changes mark edges, weak ones are removed; cv2.GaussianBlur, cv2.Laplacian.

3.1 Implementation

Three edge detectors and the boundary score

```
def edge_sobel(bgr, pct=92):
    """A. Sobel gradient magnitude, thresholded at the 92nd percentile."""
    g = _grey(bgr).astype(np.float32); mag = cv2.magnitude(cv2.Sobel(g, cv2.CV_32F, 1, 0,
ksize=3), cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=3))
    return mag > np.percentile(mag, pct)


def edge_canny(bgr, lo=None, hi=None):
    """B. Canny with automatic thresholds (hi = Otsu level of the grey image, lo = hi/2)."""
    g = _grey(bgr).astype(np.uint8)
    if hi is None: hi, _ = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU); hi =
0.5 * hi
    if lo is None: lo = 0.5 * hi
    return cv2.Canny(g, lo, hi) > 0


def edge_log(bgr, sigma=2.0, k=0.35):
    """C. Laplacian-of-Gaussian zero crossings, weak crossings suppressed."""
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY).astype(np.float32); b = cv2.GaussianBlur(g, (0,
0), sigma); lap = cv2.Laplacian(b, cv2.CV_32F, ksize=3)
    zc = np.zeros(lap.shape, bool); s = np.sign(lap)
    zc[:-1, :] |= (s[:-1, :] * s[1:, :] < 0); zc[:, :-1] |= (s[:, :-1] * s[:, 1:] < 0)
    gx = cv2.Sobel(b, cv2.CV_32F, 1, 0); gy = cv2.Sobel(b, cv2.CV_32F, 0, 1); mag =
cv2.magnitude(gx, gy)
    return zc & (mag > np.percentile(mag, 90) * k * 2)


def edge_scores(pred, gt, bbox, tol=2, pad=20):
    """Precision/recall of edge pixels within a window around the sign (tolerance tol
pixels)."""
    x0, y0, x1, y1 = bbox; sl = (slice(max(y0 - pad, 0), y1 + pad), slice(max(x0 - pad, 0), x1
+ pad))
    p = pred[sl]; g = gt[sl]
    dg = cv2.distanceTransform((~g).astype(np.uint8), cv2.DIST_L2, 3); dp =
cv2.distanceTransform((~p).astype(np.uint8), cv2.DIST_L2, 3)
    prec = (dg[p] <= tol).mean() if p.any() else 0.0; rec = (dp[g] <= tol).mean() if g.any()
else 0.0
    f1 = 2 * prec * rec / max(prec + rec, 1e-9); return prec, rec, f1
```

Evaluation rule. The ground truth is the one-pixel outer boundary of the sign (the parts hidden by branches are excluded). Edge pixels are scored in a window around the sign (the sign box plus 20 px) with a 2 px tolerance: recall is the fraction of true boundary pixels with a detected edge within 2 px, precision is the fraction of detected edge pixels within 2 px of the true boundary. Because the ground truth contains only the outer boundary, edges of the digits, the bar or the pedestrian inside the sign, and edges of trees next to the sign, count as false positives; the absolute precision is therefore low for every detector and the **ranking** is what matters.

3.2 Results on the Test Scenes

Table 17: Boundary detection on enhanced test images (mean over all nine conditions)

Detector	Precision	Recall	F1	Edge pixels (% of image)	Time (ms)
A. Sobel	0.19	0.91	0.31	8.0	3.2
B. Canny	0.24	0.86	0.37	2.8	1.4
C. LoG zero-crossing	0.22	0.87	0.34	2.7	3.9

Table 18: Boundary F1 per condition (A = Sobel, B = Canny, C = LoG)

Condition	F1 A	F1 B	F1 C
normal	0.35	0.42	0.39
sun	0.37	0.45	0.42
night	0.32	0.33	0.34
rain	0.28	0.33	0.28
shadow	0.33	0.42	0.37
blur	0.25	0.26	0.25
faded	0.33	0.39	0.38
occluded	0.27	0.34	0.31
damaged	0.32	0.38	0.36

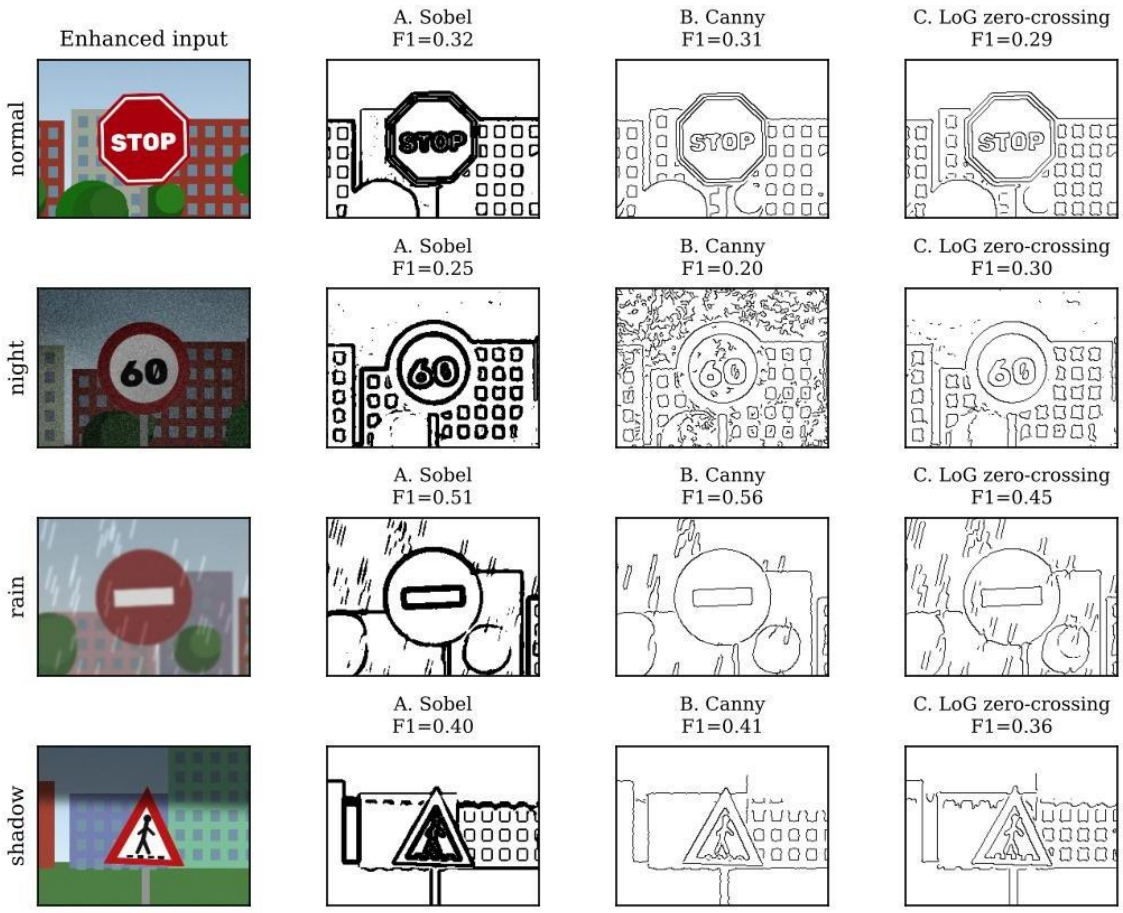


Figure 12: Edge maps of the three detectors for four scenes, with the boundary F1 of each

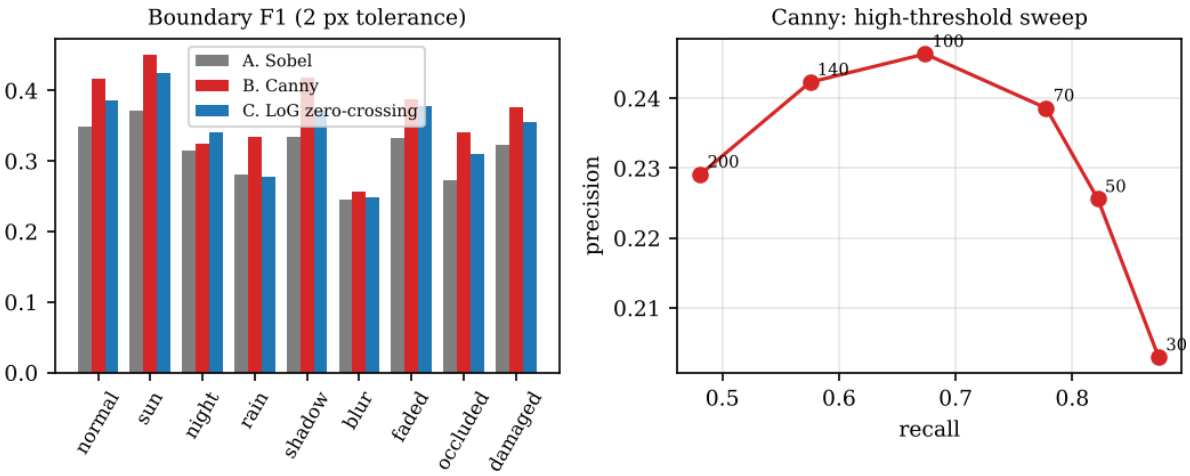


Figure 13: Boundary F1 per condition (left) and Canny precision–recall as the high threshold changes (right)

From these results I can see:

- **All three find the boundary** (recall 0.91, 0.86 and 0.87), so the difference is in the false edges.
- **Sobel marks 8.0 % of the image as edge**, about three times more than Canny (2.8 %), because it has no thinning: every edge is several pixels thick. Its F1 is the lowest (0.31).
- **Canny has the best F1 (0.37)**, the thinnest edges and the lowest time (1.4 ms). LoG is close in F1 (0.34) but 2–3 times slower and gives closed, noisy contours in clutter.
- **Motion blur is the hardest condition** for every detector (F1 about 0.26) and night and rain are next, because a blurred or hazy edge has a weak gradient.

3.3 Canny Parameters

Table 19: Canny high-threshold sweep (low = high/2)

High threshold	Precision	Recall	F1
30	0.20	0.87	0.33
50	0.23	0.82	0.35
70	0.24	0.78	0.36
100	0.25	0.67	0.35
140	0.24	0.58	0.33
200	0.23	0.48	0.30

A low threshold gives high recall (0.87) with many false edges; a high threshold loses the boundary (0.48 recall). F1 peaks around a high threshold of 70 (0.36), close to the automatic Otsu-based value used above. The Gaussian smoothing σ matters too: F1 was 0.31, 0.33, 0.34 and 0.32 for σ = 0.8, 1.4, 2.0 and 3.0, so σ between 1.4 and 2.0 is best and I keep 1.4.

3.4 Important Region Extraction

The important region of a sign is its bounding box and its shape. Three methods were compared on the same enhanced images: (A) close the Canny edges and keep the best closed contour, (B) the Hough circle transform (circular signs only), (C) the contour of the HSV mask from Task 2. A result counts as successful when the box IoU with the true box is at least 0.5. The shape is read from the number of polygon vertices (triangle 3, octagon 7–9, circle by circularity).

Three region extraction methods

```
def shape_of(cnt):
    """Classify a contour: triangle (3 vertices), octagon (7-9), circle (circularity > 0.85),
else other."""
    a = cv2.contourArea(cnt); per = cv2.arcLength(cnt, True); circ = 4 * np.pi * a / max(per *
per, 1)
```

```
    n = len(cv2.approxPolyDP(cnt, 0.03 * per, True))
    if n == 3: return 'triangle'
    if circ > 0.88 and n > 6: return 'circle'
    if 7 <= n <= 9: return 'octagon'
    return 'other'


def roi_from_edges(edges):
    """A. Close the Canny edges, fill the closed contours and keep the best sign-like one."""
    e = cv2.dilate(edges.astype(np.uint8) * 255, K5); e = cv2.morphologyEx(e, cv2.MORPH_CLOSE,
np.ones((7, 7), np.uint8))
    return pick_sign(fill_holes(e))


def roi_hough(bgr):
    """B. Hough circle transform (finds circular signs only)."""
    g = cv2.medianBlur(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY), 5)
    c = cv2.HoughCircles(g, cv2.HOUGH_GRADIENT, 1, 50, param1=120, param2=28, minRadius=18,
maxRadius=75)
    m = np.zeros(g.shape, np.uint8)
    if c is not None: x, y, r = c[0][0]; cv2.circle(m, (int(x), int(y)), int(r), 255, -1)
    return m > 0


def roi_from_mask(bgr):
    """C. Region taken directly from the HSV segmentation mask of Task 2."""
    return seg_hsv(bgr)
```

Table 20: Region extraction on the test scenes

Method	Mean box IoU	Success (IoU ≥ 0.5)	Shape correct	Time (ms)
A. Canny + closed contour	0.03	4 %	8 %	1.9
B. Hough circle	0.53	68 %	59 %	12.3
C. HSV mask contour	0.55	57 %	33 %	1.9

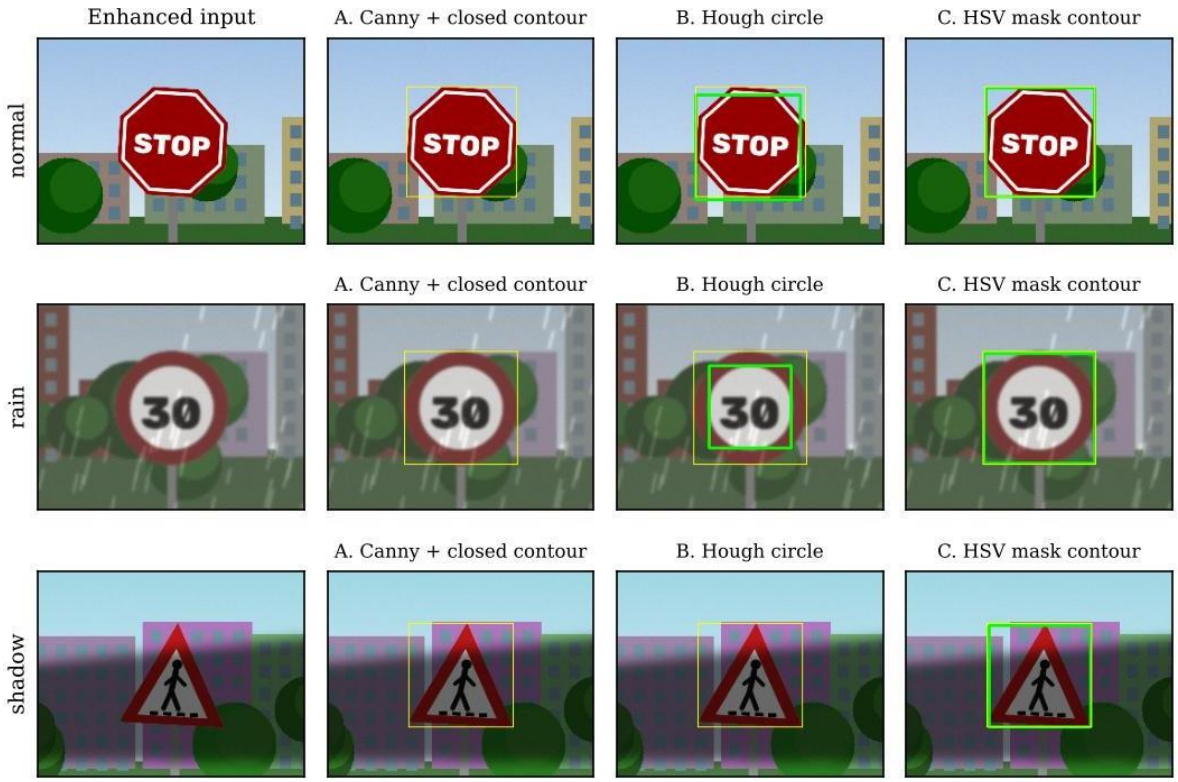


Figure 14: Detected boxes (green) against the true box (yellow) for three scenes

Method A fails in this setting: half of the 135 scenes returned no region at all (69 empty results) and the others almost never matched (4 % success). Closing the edges of the whole cluttered frame joins the sign with trees and buildings, so the closed contour is far too large or not closed; I did not test a localised version. **Method B** is successful in 68 % of the scenes, but it can only return circles, its "shape correct" figure counts only circular signs, and it is the slowest (12.3 ms). **Method C** has the highest mean IoU (0.55) and the lowest cost, but it inherits all the failures of Task 2 (faded, hidden, sun), and my polygon-vertex shape classifier was right in only 33 % of the cases, so I do not rely on it for recognition.

3.5 Final Decision for Task 3

For the **boundary** I select **B. Canny** ($\sigma = 1.4$, thresholds from the Otsu level): best F1 (0.37), thinnest edges, and the fastest (1.4 ms). For the **important region** I select **C. the contour of the HSV mask**, because it gives the best mean IoU (0.55), works for every shape rather than only circles, and costs about 1.9 ms; the Canny edges are then used only inside this region to refine the outline. **Limits:** no region method was reliable on all scenes (success at most 68 %), the shape classifier is weak, and edge scores are computed against an outer-boundary-only ground truth.

TASK 4 – Detection of Damaged, Faded and Partially Hidden Signs [10 Marks]

A sign that is faded, covered by branches or peeled and scratched can still be found by the segmentation of Task 2, but it cannot be trusted by the recognition stage. The vision system must therefore decide, for every detected sign, whether it is **healthy**, **faded**, **partly hidden** or **damaged**, and tell the city which signs need maintenance or a second look. This task compares three ways of making that decision: colour statistics, template comparison and shape completeness.

Table 21: Candidate techniques for defect detection

Technique	Basic idea and key OpenCV functions
A. Colour statistics	Look at the ring where the sign should be saturated red. Measure its mean saturation, the share of red pixels, the brightness spread and the share that is not red (cvtColor to HSV, inRange-style thresholds, erode).
B. Template comparison	Compare the 64 × 64 crop with a clean template of the best-matching sign class: normalised correlation, a standardised difference map and the Lab colour distance (matchTemplate-style correlation, absdiff, connectedComponentsWithStats).
C. Shape completeness	Extract the red region, fill it, and measure how complete it is: area ratio, solidity, convexity-defect depth, concave area and number of fragments (findContours, convexHull, convexityDefects).

4.1 Problem Definition and Assumptions

Defect classes. The nine conditions of Table 1 are mapped to four labels: the three defect conditions (faded, hidden, damaged) keep their names, and the six other conditions (normal, sun, night, rain, shadow and blur) are labelled **ok**, because lighting and motion are not defects of the sign itself. A good defect detector must therefore ignore lighting and blur and react only to the sign. This makes the task harder than it looks: the ok class contains 90 of the 135 test scenes and a detector that confuses a sunlit sign with a faded one would flood the maintenance office with false alarms.

Assumption about the sign location. Tasks 2 and 3 do not find every sign (HSV finds 58 % of the scenes, the Hough circle transform 68 %). To measure the defect detectors on their own, without mixing in the errors of the detector, I assume that the sign box is known and give every technique the same 64 × 64 crop of the enhanced image taken from the true box. Section 4.5 then shows what happens when the box is displaced, and Section 5.6 measures the whole chain with a real detector.

Success criteria (my assumption, because the scenario gives none). The city must trust the alarms, so I require an overall accuracy of at least 0.90, a false-alarm rate on healthy signs of at most 5 %, a recall of at least 0.80 for each defect class, and a decision time of at most 5 ms per sign.

Table 22: Data used in Task 4 and success criteria

Item	Value
Calibration set (variants 0–2)	135 scenes: 90 ok, 15 faded, 15 hidden, 15 damaged. Used only to train the decision rules and choose the tree depth.
Test set (variants 3–5)	135 scenes with the same class balance (90 / 15 / 15 / 15). Every table below is computed on it.
Input of each technique	Enhanced image (Task 1 rule), true sign box, 64 × 64 crop (INTER_AREA).
Decision rule	A small decision tree (CART, class-balanced) on the 5–6 numbers each technique produces; depth chosen by 5-fold cross-validation on the calibration set only.
Targets	Accuracy ≥ 0.90; false alarms on ok ≤ 5 %; recall of each defect ≥ 0.80; time ≤ 5 ms.

4.2 Implementation

The three techniques share two helpers: a function that cuts the 64 × 64 crop and a function that finds the clean template with the highest correlation. Each technique returns a short feature vector and a defect map (a boolean image of the pixels it considers damaged). The same decision-tree learner turns each feature vector into a label, so differences in the results come from the features and not from the classifier.

```
def crop_of(img, bbox, jit=(0, 0)):
    x0, y0, x1, y1 = bbox; dx, dy = jit
    x0, x1 = max(x0 + dx, 0), min(x1 + dx, W); y0, y1 = max(y0 + dy, 0), min(y1 + dy, H)
    return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)


def best_template(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY).astype(np.float32); best = ('', -2.0)
    for k, t in TG.items():
        m = TEMPL[k][1]
        a = g[m] - g[m].mean(); b = t[m] - t[m].mean(); r = (a * b).sum() / (np.linalg.norm(a) *
np.linalg.norm(b) + 1e-6)
        if r > best[1]: best = (k, float(r))
    return best
```

Technique A measures the colour of the ring. The ring is the outer band of the expected sign shape (the template mask minus nine erosions). A healthy sign has a ring that is almost completely saturated red; a faded sign has a pale ring; a covered sign has green pixels in the ring; a peeled sign has grey patches in it.

```
def tech_A(crop):
    k, _ = best_template(crop); m = TEMPL[k][1]
    ring = m & ~(cv2.erode(m.astype(np.uint8), np.ones((3, 3), np.uint8), iterations=9) > 0)
    h, s, v = cv2.split(cv2.cvtColor(crop, cv2.COLOR_BGR2HSV))
    red = ((h <= 10) | (h >= 170)) & (s >= 90) & (v >= 60)
    notred = ring & ~red
    feats = [s[ring].mean(), red[ring].mean(), v[ring].std(), s[m].mean(), notred.sum() /
max(ring.sum(), 1)]
    return np.array(feats, np.float32), notred
```

Technique B compares the crop with the template. Both are standardised inside the sign mask (zero mean, unit variance), so the comparison ignores overall brightness and contrast. Pixels whose standardised difference exceeds 1.3 are marked, small specks are removed, and the largest blob is measured. The Lab colour distance to the template is added because standardising the grey levels would otherwise hide fading completely.

```
def tech_B(crop):
    k, ncc = best_template(crop); m = TEMPL[k][1]
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY).astype(np.float32); t = TG[k]
    gn = (g - g[m].mean()) / (g[m].std() + 1e-6); tn = (t - t[m].mean()) / (t[m].std() + 1e-6)
    d = np.abs(gn - tn) * m
    mp = (d > 1.3).astype(np.uint8); mp = cv2.morphologyEx(mp, cv2.MORPH_OPEN, np.ones((3, 3),
np.uint8))
    n, lbl, st, _ = cv2.connectedComponentsWithStats(mp)
    big = st[1:, cv2.CC_STAT_AREA].max() if n > 1 else 0
    colour = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB).astype(np.float32); tc = cv2.cvtColor(TEMPL[k][0],
cv2.COLOR_BGR2LAB).astype(np.float32)
    dc = np.linalg.norm(colour - tc, axis=2)[m].mean()
    feats = [ncc, mp.sum() / m.sum(), big / m.sum(), d[m].mean(), dc]
    return np.array(feats, np.float32), mp > 0
```

Technique C ignores colour detail and asks whether the red region still has the full shape of a sign. It thresholds red hue with a relaxed saturation, fills holes, and measures how much of the convex hull is empty (concave area), how deep the deepest convexity defect is, and how many separate fragments exist.

```
def tech_C(crop):
    h, s, v = cv2.split(cv2.cvtColor(crop, cv2.COLOR_BGR2HSV))
    red = ((h <= 12) | (h >= 168)) & (s >= 50) & (v >= 40)
    m = red.astype(np.uint8) * 255; m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((3, 3),
np.uint8))
    f = fill_holes(m); cs, _ = cv2.findContours(f, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not cs: return np.array([0, 0, 0, 0, 0, 0], np.float32), np.zeros((S, S), bool)
    c = max(cs, key=cv2.contourArea); a = cv2.contourArea(c); hull = cv2.convexHull(c); ha =
cv2.contourArea(hull)
    hm = np.zeros((S, S), np.uint8); cv2.drawContours(hm, [hull], -1, 255, -1)
    conc = (hm > 0) & (f == 0)
    depth = 0.0
    hi = cv2.convexHull(c, returnPoints=False)
    if hi is not None and len(hi) > 3:
        try:
            df = cv2.convexityDefects(c, hi); depth = 0.0 if df is None else df.reshape(-1, 4)[:,
3].max() / 256.0 / S
        except cv2.error: depth = 0.0
    ncomp = cv2.connectedComponents(m)[0] - 1
    feats = [a / (S * S), a / max(ha, 1), depth, conc.sum() / max(ha, 1), ncomp, (f > 0).sum() / (S *
S)]
    return np.array(feats, np.float32), conc
```

4.3 Results on the Test Scenes

Each technique was run on the 135 test scenes. Table 23 gives the overall results, Table 24 the precision and recall of each class, and Table 25 the correct decisions for every condition. A fourth row, F, is a what-if in which the three feature vectors are joined and one tree is trained on all of them.

Table 23: Defect detection on the test scenes (135 scenes)

Technique	Accuracy	False alarms (ok)	Recall faded	Recall hidden	Recall damaged	Mean defect recall	Time (ms)
A. Colour statistics	0.93	2 %	1.00	0.93	0.60	0.84	0.27
B. Template comparison	0.88	12 %	1.00	1.00	0.67	0.89	0.45
C. Shape completeness	0.61	30 %	0.13	0.87	0.33	0.44	0.09
What-if F: A + B + C joined	0.90	4 %	0.87	0.93	0.53	0.78	0.80

Table 24: Precision and recall of every class (P = precision, R = recall)

Technique	ok (P / R)	faded (P / R)	hidden (P / R)	damaged (P / R)
A. Colour	0.94 / 0.98	1.00 / 1.00	1.00 / 0.93	0.75 / 0.60
B. Template	0.95 / 0.88	1.00 / 1.00	0.79 / 1.00	0.56 / 0.67
C. Shape	0.74 / 0.70	0.11 / 0.13	0.93 / 0.87	0.28 / 0.33

Table 25: Correct decisions per condition (A = colour, B = template, C = shape; 15 scenes each)

Condition	A	B	C	True label
normal	1.00	0.87	0.53	ok
sun	1.00	1.00	0.53	ok
night	1.00	1.00	1.00	ok
rain	0.87	1.00	0.73	ok
shadow	1.00	0.80	0.53	ok
blur	1.00	0.60	0.87	ok
faded	1.00	1.00	0.13	defect
hidden	0.93	1.00	0.87	defect
damaged	0.60	0.67	0.33	defect

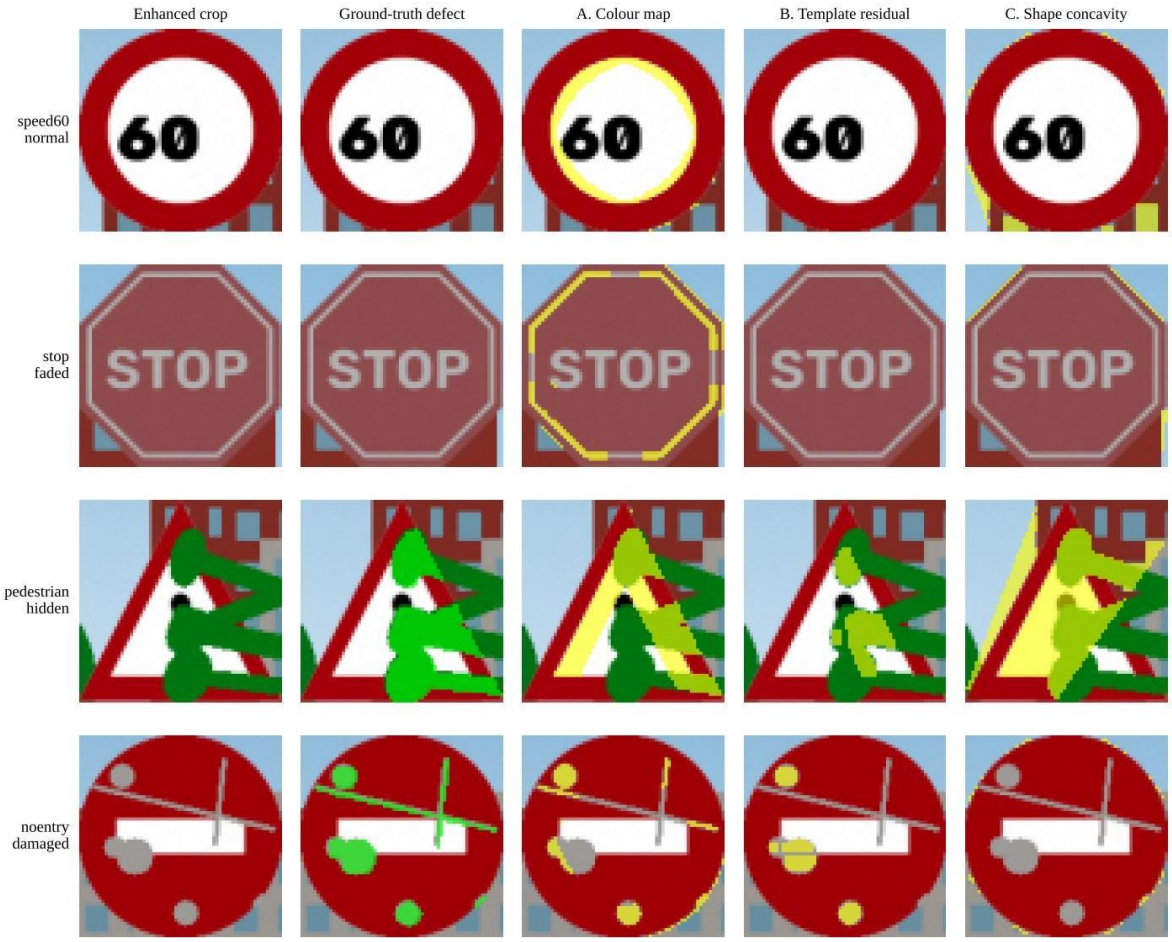


Figure 15: Enhanced crops, the true defect, and the defect maps of the three techniques for four test signs

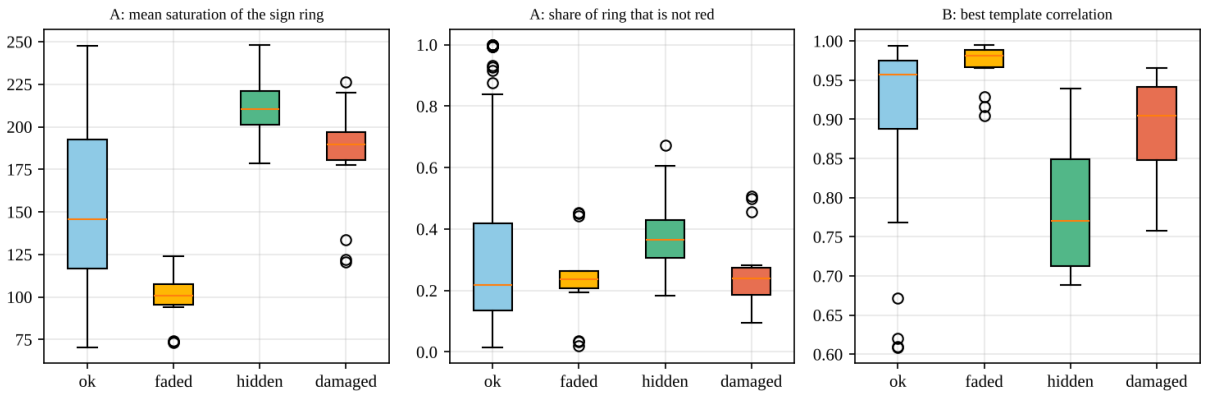


Figure 16: Distribution of three key features over the four classes of the test set

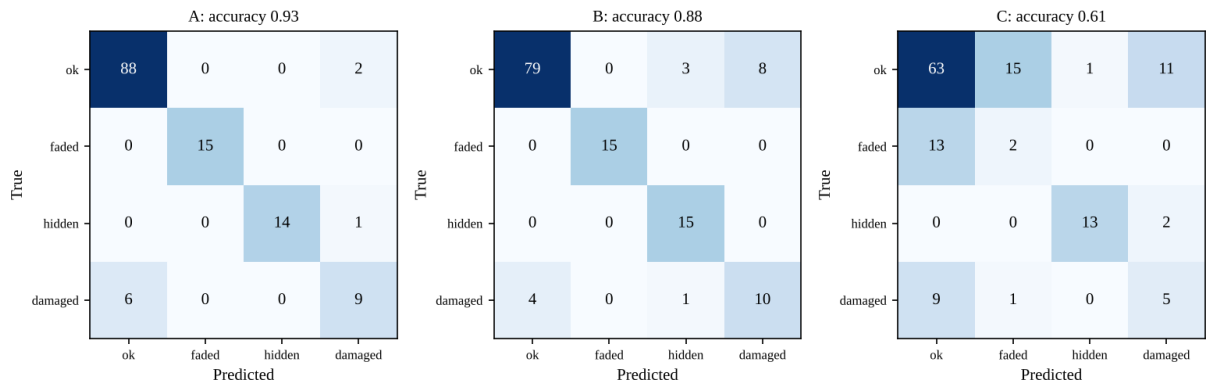


Figure 17: Confusion matrices of the three techniques on the test set

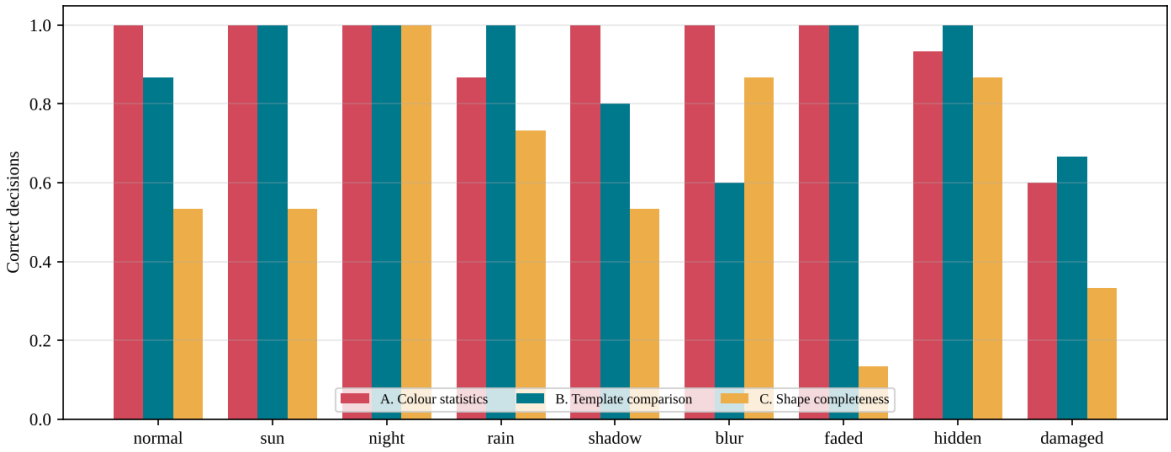


Figure 18: Correct decisions per condition for the three techniques

From these results I can see:

- **A (colour statistics) is the best technique.** It reaches an accuracy of 0.93, a false-alarm rate of only 2 % on the 90 healthy scenes and finds all faded signs (recall 1.00). Figure 16 shows why: the median saturation of the ring is 101 for faded signs against 146 for the ok group, so the faded box lies clearly below the others.
- **B (template comparison) finds every faded and hidden sign but raises too many false alarms.** Its recall for faded and hidden signs is 1.00 and 1.00, but 12 % of the healthy scenes are flagged. Table 25 shows that they come mainly from blur (0.60 correct), shadow (0.80) and normal scenes (0.87): a template difference cannot tell a smeared or shaded sign from a damaged one.
- **C (shape completeness) is the weakest.** The accuracy is only 0.61 and 30 % of healthy scenes raise an alarm. A likely cause is that red brick walls and the red vehicle touch the red

mask of many signs and change its shape, so a healthy sign looks incomplete. For faded signs C is blind (recall 0.13): the shape of a faded sign is complete, and its features equal those of healthy signs (median solidity 0.97 against 0.97). A shape test cannot reveal a colour defect.

- **Hidden signs are found by all three** (recall 0.93, 1.00 and 0.87), because green branches change both colour and outline. **Damaged signs are the hardest** (recall 0.60, 0.67 and 0.33): peeled patches of 5–11 px and 2-px scratches cover only a few percent of the ring.
- **Joining all features does not help.** The what-if F has an accuracy of 0.90, lower than A alone (0.93), probably because with only 135 training scenes a tree on 16 features starts to fit noise from the weaker features of B and C. More features are not better when the training set is small.

4.4 Locating the Defect

Deciding that a sign is defective is not enough for a maintenance team; they also want to know where. For hidden and damaged signs I compared the defect map of each technique with the true defect (branches or peeled patches) inside the sign, and measured the Dice score and the error of the estimated covered fraction of the sign (Table 26). Faded signs are excluded because the whole sign is faded and there is no region to locate.

Table 26: Localisation of hidden and damaged regions (Dice, higher is better; covered-area error, lower is better)

Technique	Dice (hidden)	Dice (damaged)	Covered-area error (hidden)
A. Colour map	0.55	0.35	0.07
B. Template residual	0.40	0.22	0.19
C. Shape concavity	0.55	0.12	0.18

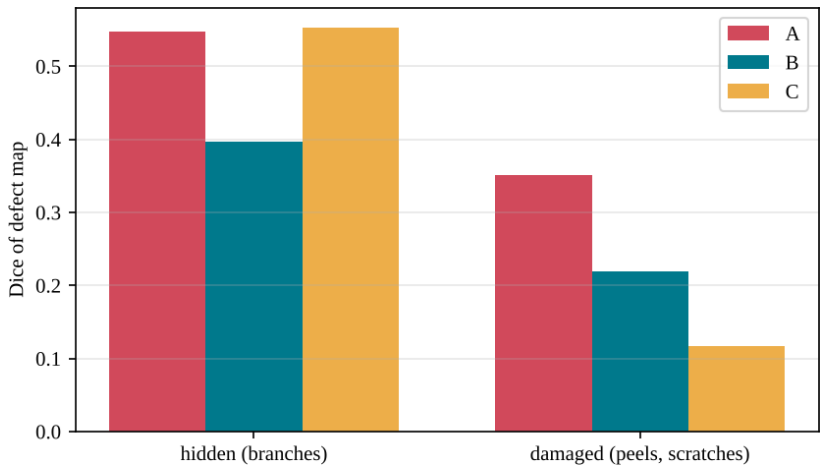


Figure 19: Dice score of the defect maps for hidden and damaged signs

The colour map of A is the most useful: it reaches a Dice of 0.55 for hidden signs and 0.35 for damaged signs, and it estimates the covered fraction with an error of 0.07 of the sign area, against 0.19 for B and 0.18 for C. C locates hidden regions as well as A (Dice 0.55), because a branch breaks the outline, but it cannot see peeled patches inside the sign (Dice 0.12), since the hull of the sign is still complete. B finds the patches but also marks every edge shifted by a few pixels. In Figure 15 the yellow overlay of A follows the pale peeled patches closely, while C marks only the corner where the branch cut the outline. None of the Dice scores is high, so these maps should be used to **point** to a region and not to measure its exact area.

4.5 Parameter Sensitivity

Two choices can change the results: the depth of the decision tree and the position of the box. The depth was chosen by 5-fold cross-validation on the calibration set only (Table 27), never on the test set, so the test accuracy in Table 23 is an honest estimate. The box position was then displaced by 0, 3, 6 and 10 pixels to the right (and half as much downwards) in the test scenes, with the trained rules unchanged (Table 28).

Table 27: Cross-validated accuracy on the calibration set against tree depth

Tree depth	2	3	4	5	6	8
A. Colour	0.50	0.58	0.72	0.77	0.81	0.83
B. Template	0.59	0.70	0.78	0.79	0.84	0.82
C. Shape	0.41	0.59	0.59	0.55	0.63	0.65

Table 28: Test accuracy when the box is displaced (pixels)

Displacement	0 px	3 px	6 px	10 px
A. Colour	0.93	0.77	0.71	0.62
B. Template	0.88	0.56	0.59	0.67
C. Shape	0.61	0.64	0.51	0.48

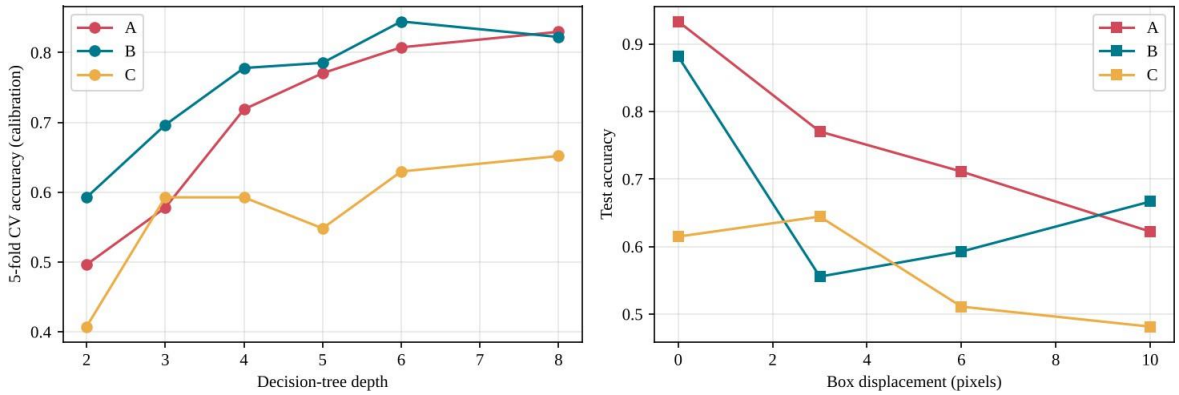


Figure 20: Cross-validated accuracy against tree depth (left) and test accuracy against box displacement (right)

Depth 8 was selected for A (CV accuracy 0.83), depth 6 for B (0.84) and depth 8 for C (0.65). The cross-validated accuracy on the calibration set is lower than the test accuracy, because every calibration fold has only 108 training scenes; this means that the choice of the depth is itself uncertain and that the test result should not be over-trusted. The displacement test is more important: when the box moves by only 3 px, the accuracy of A falls from 0.93 to 0.77, that of B falls from 0.88 to 0.56, and at 10 px they fall to 0.62 and 0.67. The reason is that the ring and the template are fixed zones of the 64 × 64 crop; if the box is shifted, the ring zone includes sky or brick and looks damaged. The defect check is therefore only as good as the box of the detector, and the box must be accurate to about 2–3 px.

4.6 What the Selected Rule Looks Like

A decision tree can be read, which is an advantage in a public-safety system. Table 29 shows the first three levels of the tree learned from the colour features of A on the calibration set (the deployed tree has depth 8 and 17 leaves). Brightness spread of the ring (ring_Vstd) is the first split: a low spread means that the ring is uniform, which is typical of healthy and faded signs, and a high spread means that something with a different brightness (a branch, a patch, a scratch) is in the ring.

Inside the uniform branch, a low sign saturation together with a high share of non-red pixels means faded.

Table 29: First three levels of the decision tree learned from the colour features (calibration set)

```
|--- ring_Vstd <= 18.89
|   |--- sign_S <= 109.73
|   |   |--- notred <= 0.66
|   |   |   |--- class: faded
|   |   |--- notred >  0.66
|   |   |   |--- class: ok
|   |--- sign_S >  109.73
|   |   |--- notred <= 0.08
|   |   |   |--- class: ok
|   |   |--- notred >  0.08
|   |   |   |--- class: damaged
|--- ring_Vstd >  18.89
|   |--- ring_S <= 198.73
|   |   |--- ring_red <= 0.78
|   |   |   |--- class: damaged
|   |   |--- ring_red >  0.78
|   |   |   |--- class: ok
|   |--- ring_S >  198.73
|   |   |--- ring red <= 0.84
|   |   |   |--- class: hidden
|   |   |--- ring_red >  0.84
|   |   |   |--- class: ok
```

4.7 Failure Analysis

Table 25 and Figure 17 show where A fails. It misses 6 of the 15 damaged test signs, which are the signs with the smallest peeled areas: a patch of 5 px radius on a 100-px sign changes only about 1 % of the ring, and the tree cannot separate this from camera noise. It also raises false alarms on 2 of the 15 rain scenes, where streaks and haze add bright pixels to the ring. These are the two weak spots that field testing must check: very small damage and heavy rain.

The what-if result is also a warning against over-reading the numbers. The test set has only 15 scenes per defect, so one wrong decision changes a recall by 0.07. The ranking A > B > C is large enough to be reliable (the gap between A and C is about 0.32 in accuracy), but the gap between A and B (0.05) is within what a different random draw of scenes could change, and I base the choice on the false-alarm rate (2 % against 12 %) and the box-shift behaviour as well as on accuracy.

4.8 Final Decision for Task 4

I select **A. colour statistics of the sign ring with a decision tree of depth 8**. Evidence: it has the highest accuracy (0.93), the lowest false-alarm rate (2 %), perfect recall for faded signs, the best localisation (Dice 0.55 for hidden regions and the smallest covered-area error, 0.07), and it needs only 0.27 ms. B is kept as a second opinion for hidden signs (recall 1.00) but not as the main rule

because of its 12 % false alarms, and C is rejected. **Limits:** damaged signs are found only 60 % of the time, so the success criterion of 0.80 recall is met for faded and hidden signs but not for damaged signs; the rule needs a box accurate to 2–3 px; and it was trained on generated scenes with red signs only, so it must be recalibrated on real photographs before the city relies on it.

TASK 5 – Recognition-Ready Output and Integrated Pipeline Design [10 Marks]

The last stage of the system hands each sign to a recognition module that decides which sign it is (STOP, speed limit 30, speed limit 60, No Entry or pedestrian crossing). A recognition module works best when every sign arrives in the same form: the same size, the same orientation, the same brightness and no distracting background. This task compares three ways of producing such a normalised output, measures how well a recogniser reads each of them, and then joins all five tasks into one pipeline whose stages are timed and whose design is defended.

Table 30: Candidate normalisation techniques

Technique	Basic idea and key OpenCV functions
A. Bounding-box crop	Cut the box found by the detector and resize it to 64 × 64 (array slicing, resize with INTER_AREA). No other change.
B. Mask-guided standardisation	Cut the box with an 8 % margin, replace everything outside the sign mask by neutral grey, and standardise each colour channel inside the mask to the same mean and spread, then resize to 64 × 64.
C. Contour-aligned normalisation	Find the minimum-area rectangle of the sign contour, rotate the image to remove its tilt, grey out the background, apply CLAHE to the luminance and pad to a square without changing the aspect ratio (minAreaRect, warpAffine, createCLAHE).

5.1 What Recognition-Ready Output Means

The output of the vision system is not only a picture. For every frame the pipeline returns one record per sign with the normalised crop and the information gathered in Tasks 1–4, so that the recognition module, the maintenance office and the driver-assistance logic can each use what they need (Table 31).

Table 31: Output record produced for every detected sign

Field	Content	Produced by
crop	64 × 64 × 3 normalised image (uint8, BGR)	Task 5 normalisation
bbox	x0, y0, x1, y1 in the 640 × 480 frame	Task 2 segmentation
mask / contour	Boolean sign mask and its outline	Tasks 2 and 3
shape	triangle, circle or octagon (from the contour)	Task 3 shape test
defect	ok, faded, hidden or damaged	Task 4 colour statistics
route	hsv, hsv-raw or hough (which detector found it)	Task 5 pipeline
label, score	Sign class and classifier score from the recogniser	Task 5 recogniser

5.2 Implementation

The three normalisation functions take the frame, the box and the mask and return a 64 × 64 colour crop. To measure their effect on recognition, every crop is turned into a feature vector (histogram of oriented gradients of the grey image plus a 4 × 4 grid of Lab colours) and classified by a support vector machine with an RBF kernel. A separate recogniser is trained for each normalisation, using the 135 calibration scenes (true box and mask) so that each technique is judged with a recogniser that was trained on its own kind of input.

```
def norm_A(img, bbox, mask):
    """A. Plain bounding-box crop resized to 64 x 64."""
    x0, y0, x1, y1 = bbox; return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)


def _margin_box(bbox, mask, m=0.08):
    x0, y0, x1, y1 = bbox; pw = int((x1 - x0) * m); ph = int((y1 - y0) * m)
    return max(x0 - pw, 0), max(y0 - ph, 0), min(x1 + pw, W), min(y1 + ph, H)


def norm_B(img, bbox, mask):
    """B. Mask-guided crop: background outside the sign replaced by neutral grey, colour/brightness
standardised inside the mask."""
    x0, y0, x1, y1 = _margin_box(bbox, mask); im = img[y0:y1, x0:x1].astype(np.float32); m =
mask[y0:y1, x0:x1]
    if m.sum() < 20: return cv2.resize(img[bbox[1]:bbox[3], bbox[0]:bbox[2]], (S, S),
interpolation=cv2.INTER_AREA)
    out = np.full_like(im, 128.0)
    for c in range(3):
        v = im[..., c]; mu, sd = v[m].mean(), v[m].std() + 1e-3
        out[..., c][m] = np.clip(128 + (v[m] - mu) * 55.0 / sd * 0.6 + (mu - 128) * 0.0, 0, 255)
    return cv2.resize(out.astype(np.uint8), (S, S), interpolation=cv2.INTER_AREA)


def norm_C(img, bbox, mask):
    """C. Contour-aligned: de-skew with the minimum-area rectangle, grey background, CLAHE on L,
aspect-preserving pad to 64 x 64."""
    m8 = mask.astype(np.uint8); cs, _ = cv2.findContours(m8, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if not cs: return norm_A(img, bbox, mask)
    c = max(cs, key=cv2.contourArea); (cx, cy), (rw, rh), ang = cv2.minAreaRect(c)
    ang = ang % 90.0; ang = ang - 90.0 if ang > 45 else ang                # smallest rotation that
aligns the box
    M = cv2.getRotationMatrix2D((cx, cy), ang, 1.0)
    ri = cv2.warpAffine(img, M, (W, H), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE); rm
= cv2.warpAffine(m8, M, (W, H), flags=cv2.INTER_NEAREST) > 0
    bb = bbox_of(rm)
    if bb is None: return norm_A(img, bbox, mask)
    x0, y0, x1, y1 = _margin_box(bb, rm, 0.05); im = ri[y0:y1, x0:x1].copy(); m = rm[y0:y1, x0:x1]
    lab = cv2.cvtColor(im, cv2.COLOR_BGR2LAB); lab[..., 0] = cv2.createCLAHE(2.0, (4,
4)).apply(lab[..., 0]); im = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
    im[~m] = 128; h, w = im.shape[:2]; sc = S / max(h, w)
    im = cv2.resize(im, (max(1, int(round(w * sc))), max(1, int(round(h * sc)))),
interpolation=cv2.INTER_AREA)
    out = np.full((S, S, 3), 128, np.uint8); oy = (S - im.shape[0]) // 2; ox = (S - im.shape[1]) // 2
    out[oy:oy + im.shape[0], ox:ox + im.shape[1]] = im; return out
```

```
def feat(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY); h = _hog(g, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2)).ravel()
    lab = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB).astype(np.float32); grid = cv2.resize(lab, (4, 4),
interpolation=cv2.INTER_AREA).ravel() / 255.0
    return np.concatenate([h, grid]).astype(np.float32)


def detect(img):
    e = enhance(img); m = seg_hsv(e); how = 'hsv'
    if not m.any(): m = seg_hsv(img); how = 'hsv-raw'
    if not m.any(): m = roi_hough(e); how = 'hough'
    if not m.any(): return None, None, 'none'
    return bbox_of(m), m, how
```

The function detect() is the segmentation stage selected in Tasks 1–3. It enhances the frame (gamma, or CLAHE for dark frames), runs the adaptive HSV segmentation, falls back to the HSV segmentation on the raw frame, and finally to the Hough circle transform. If all three fail the frame returns no sign.

5.3 Three Experiments

Three experiments separate the causes of error. **(i) Oracle region:** the true box and mask are given to the normalisation, so that only the normalisation is being compared. **(ii) Full pipeline:** the box and mask come from detect(), so that errors of the detector are included; a frame with no detection counts as a wrong answer. **(iii) Tilt test:** the oracle-region test is repeated after rotating every scene by +8°, +16° and −16° around the sign, with the recognisers unchanged, to see which normalisation survives a camera that is not level.

5.4 Results on the Test Scenes

Table 32: Recognition accuracy of the three normalisations (135 test scenes)

Technique	Oracle region	Full pipeline	Pipeline, signs found only	Mean over tilts	Time (ms)
A. Box crop	0.99	0.63	0.92	0.87	0.06
B. Mask + standardise	0.96	0.61	0.89	0.71	0.40
C. Deskew + CLAHE	0.94	0.54	0.81	0.87	4.07

Table 33: Recognition accuracy per condition with the oracle region (15 scenes each)

Condition	A	B	C
normal	1.00	1.00	1.00
sun	1.00	1.00	1.00
night	1.00	0.93	1.00
rain	1.00	1.00	1.00
shadow	1.00	1.00	0.93
blur	0.93	1.00	1.00
faded	1.00	1.00	0.93
hidden	1.00	0.73	0.80
damaged	0.93	0.93	0.80

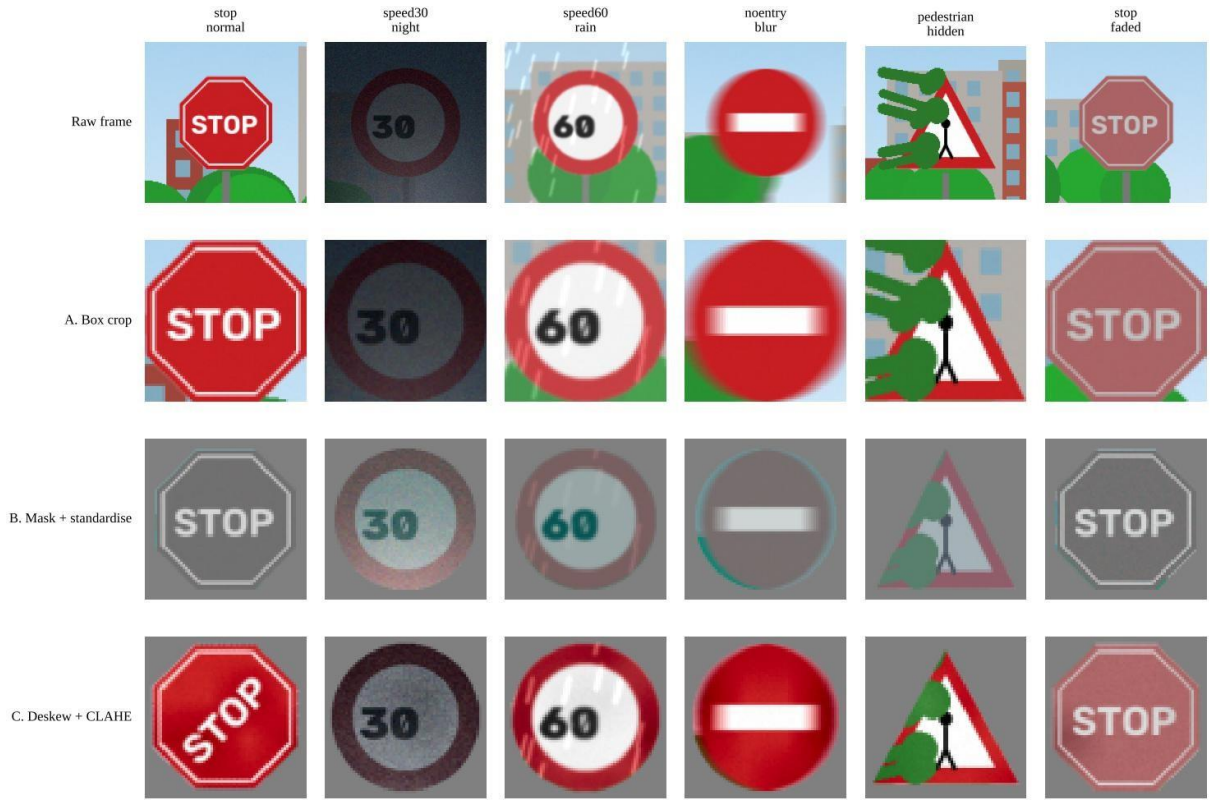


Figure 21: Six test signs after the three normalisations (first row: raw frame region)

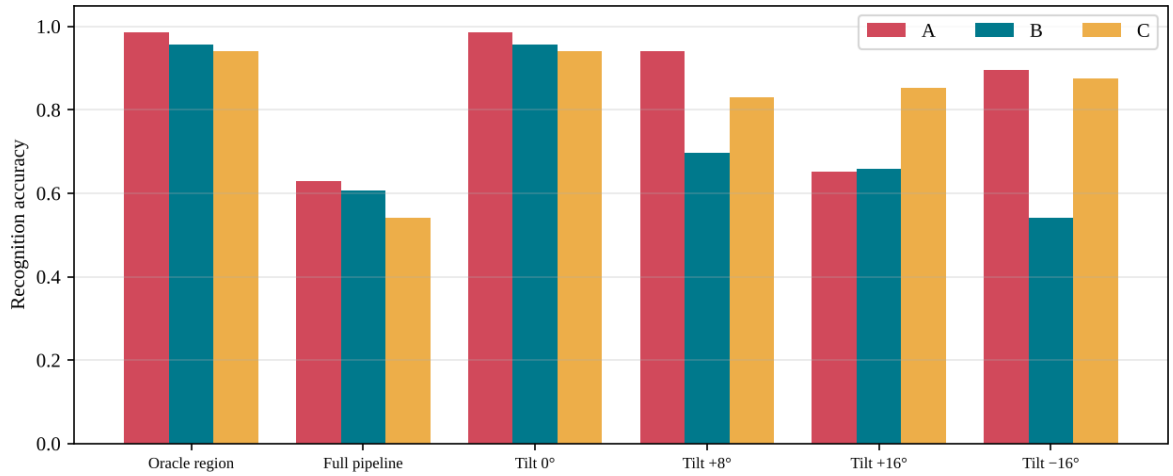


Figure 22: Recognition accuracy of the three normalisations in the oracle, pipeline and tilt experiments

From these results I can see:

- **With a correct region, all three normalisations are accurate** (A 0.99, B 0.96, C 0.94), and the plain box crop is the best. The recogniser was trained on the same kind of crops, and HOG features are already insensitive to small brightness changes, so the extra standardisation

of B and CLAHE of C bring no gain on upright signs. They even lose a little: B grey-fills the hidden signs badly (0.73 for hidden), probably because standardising the colours inside a mask that includes green branches changes the colour of the red ring.

- **With a real detector the accuracy falls to 0.63 for A** (B 0.61, C 0.54). The drop is not caused by the normalisation: when the sign is found correctly, A reads it right 92 % of the time. The cause is detection: only 67 % of the 135 scenes give a region that overlaps the true sign with IoU ≥ 0.5.
- **Normalisation C is the most robust to tilt.** When the scene is rotated by +16°, A falls from 0.99 to 0.65 and B to 0.66, while C keeps 0.85. The mean over the four tilt settings is 0.87 for C, 0.87 for A and 0.71 for B, so C and A are equal on average, but C has a far better worst case (Table 34).
- **Cost.** A needs 0.06 ms, B 0.40 ms and C 4.07 ms. C is about 65 times slower than A because it rotates the whole frame, but even C is only 4 ms and fits the 33-ms frame budget.

Table 34: Recognition accuracy when the scene is rotated (oracle region)

Rotation	A. Box crop	B. Mask + standardise	C. Deskew + CLAHE
0°	0.99	0.96	0.94
+8°	0.94	0.70	0.83
+16°	0.65	0.66	0.85
-16°	0.90	0.54	0.87

Note that the tilt of a real road camera is usually small because the camera is fixed to the vehicle, but signs themselves can lean after an accident or a storm, and vehicles roll on curves. The test therefore indicates how much safety margin each normalisation has, and C is the only one that keeps an accuracy of 0.83 or more in all rotations. The rectangle angle of C is probably less reliable for circles and octagons, which look almost the same after rotation, so its gain should be checked per sign type on real data.

5.5 Integrated Pipeline Design

Figure 23 joins the five tasks. The frame is enhanced (Task 1), the sign is segmented (Task 2), its boundary is found (Task 3), the sign is checked for defects (Task 4) and finally normalised and recognised (Task 5). Every stage was timed on one CPU core, as the mean of the 135 test frames, with the selected technique of each task.

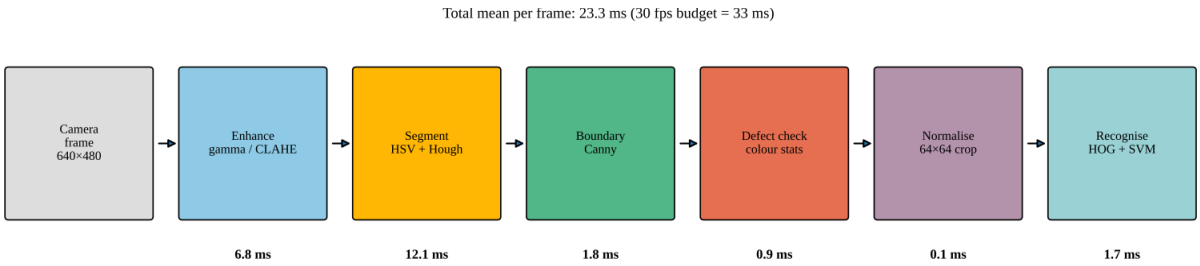


Figure 23: Block diagram of the integrated pipeline with the measured mean time of each stage

Table 35: Stage times of the integrated pipeline (mean of 135 test frames, one CPU core)

Stage	Mean time (ms)	Share of total
Enhancement (Task 1: gamma, CLAHE if dark)	6.78	29 %
Segmentation (Task 2: HSV, then Hough)	12.11	52 %
Boundary (Task 3: Canny)	1.82	8 %
Defect check (Task 4: colour statistics)	0.86	4 %
Normalisation (Task 5: box crop)	0.09	0 %
Recognition (HOG + SVM)	1.68	7 %
Total per frame	**23.3**	100 %

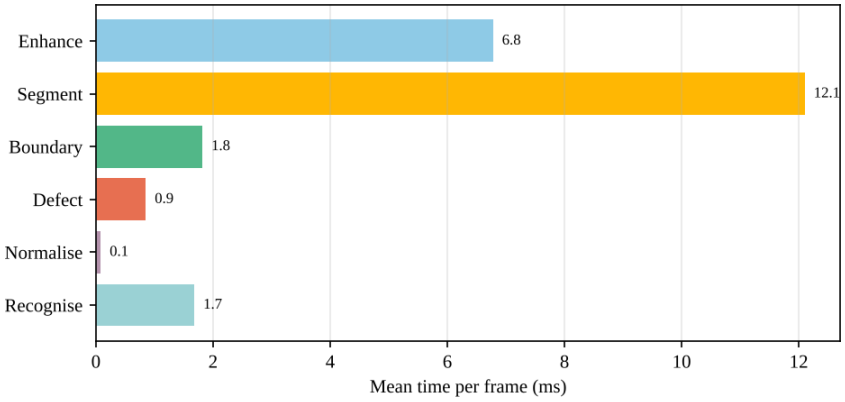


Figure 24: Mean time of each pipeline stage

The pipeline needs 23.3 ms per frame on one CPU core, which is below the 33 ms that 30 frames per second allow and far below the 100 ms limit assumed in Table 1. Segmentation is the most expensive stage (52 %), because the Hough fallback is run when HSV finds nothing, and enhancement is second (29 %). The defect check, the normalisation and the recogniser together need only 2.6 ms, so the Task 4 and Task 5 additions do not threaten the real-time target. The timing excludes image capture and display.

5.6 Where the Pipeline Fails: Detection

Table 36 and Figure 25 show the detection rate of the first two stages for every condition. The region is counted as correct if its IoU with the true sign is at least 0.5. Of the 135 scenes, 65 are found by HSV on the enhanced frame, 6 by HSV on the raw frame, 51 by the Hough circle fallback and 13 return nothing; 90 of these regions are correct.

Table 36: Share of scenes in which the sign region is found correctly (IoU ≥ 0.5), 15 scenes per condition

Condition	normal	sun	night	rain	shadow	blur	faded	hidden	damaged
Sign found	1.00	0.67	0.53	1.00	1.00	0.27	0.40	0.40	0.73

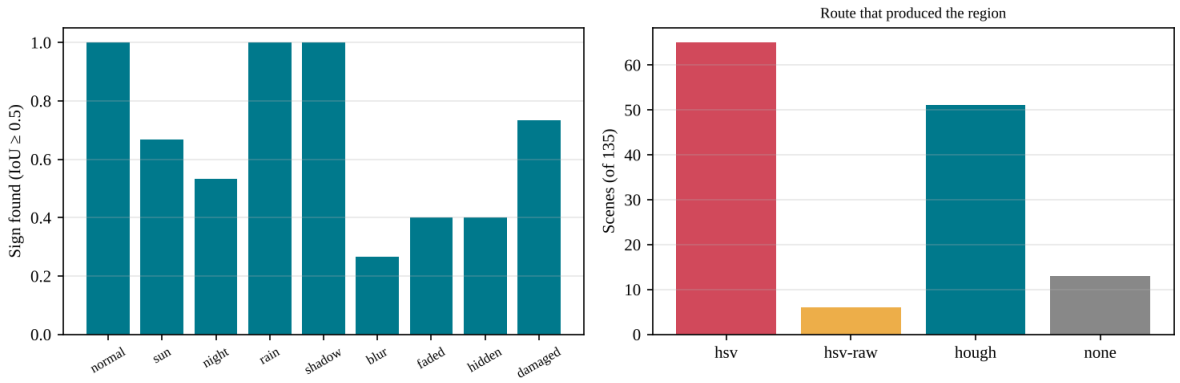


Figure 25: Detection rate per condition (left) and route that produced the region (right)

Normal, rain and shadow scenes are found in 100 % of the cases, but blur (0.27), faded signs (0.40), hidden signs (0.40) and night (0.53) are the weak conditions. The most likely reasons are the ones found in the earlier tasks: motion blur spreads the red over the neighbouring pixels so the saturation drops below the HSV threshold; faded signs have almost no saturation left; branches break the outline so the region fails the compactness test of pick_sign(); and at night the saturation of the red is lost in the noise. The Hough fallback finds a circle in many frames, but not always the sign: 32 scenes produce a region that does not overlap the sign, against only 13 that produce no output at all. A wrong region is more dangerous than no output, because it is passed to the recogniser as if it were a sign.

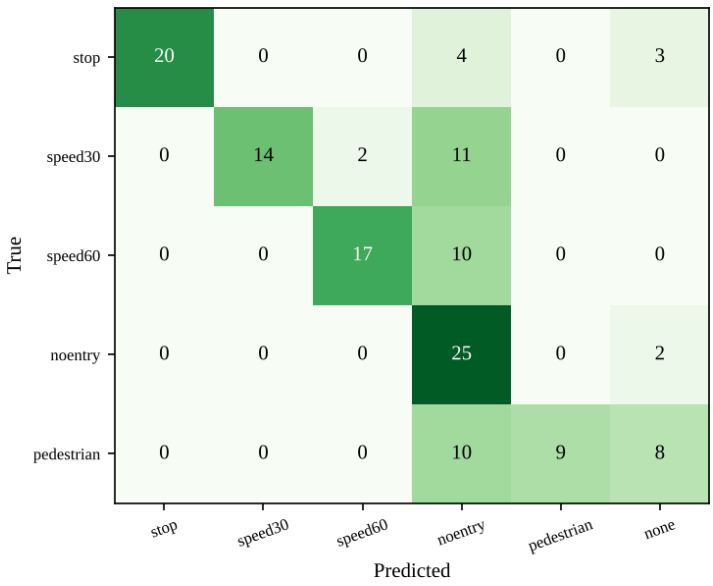


Figure 26: Confusion matrix of the full pipeline with normalisation A (rows: true sign, columns: output)

Figure 26 shows the effect on recognition. Almost all errors are in the last two columns: scenes with no sign found, or with a wrong region that the recogniser reads as No Entry (a plain red disc looks like that class). The pedestrian sign is the worst case (0.33 correct) because it is a triangle and the circle fallback cannot find it. When the region is correct, recognition is reliable: 92 % of the found signs are read correctly. The defect check in the same pipeline decides correctly for 78 % of the 122 frames that gave a region, with a false-alarm rate of 8 % on healthy signs; this is worse than the 0.93 of Table 23 because the detector boxes are not as exact as the true boxes (Section 4.5).

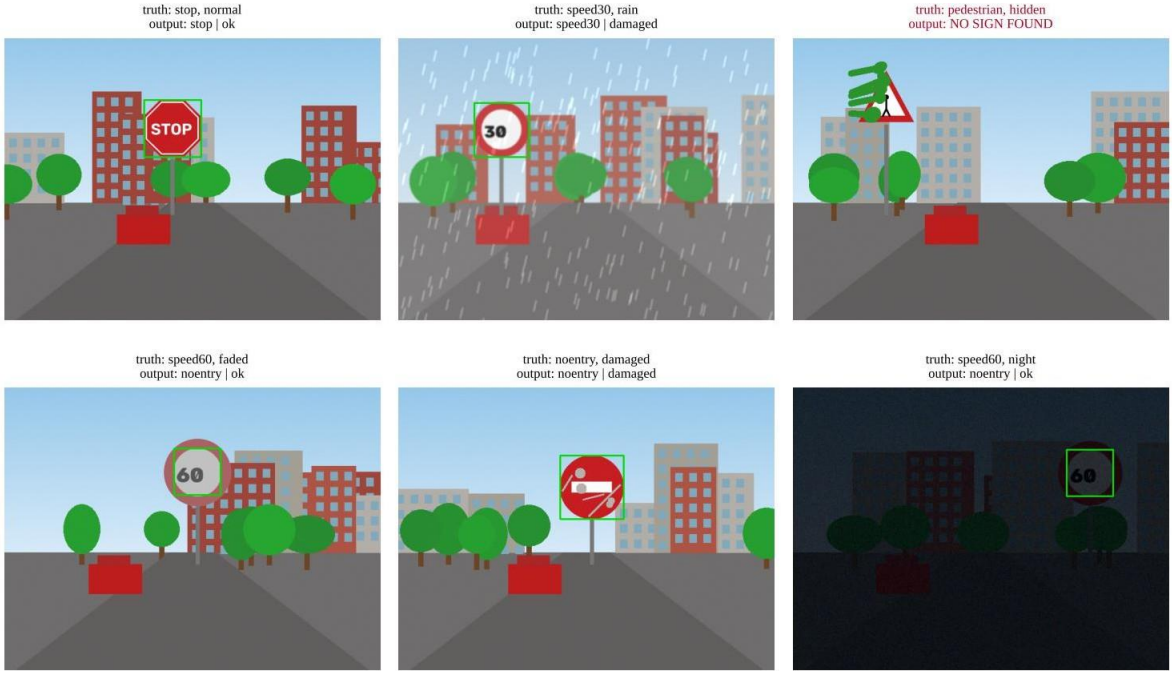


Figure 27: Output of the final pipeline for six test frames (green box; recognised sign | defect decision)

5.7 Decision Table of the Final Pipeline

Table 37: Selected technique, evidence and limit of every stage

Stage	Selected technique	Main evidence	Main limit
1 Enhancement	Gamma on L; CLAHE if median L < 60	Best mean Dice (0.55) and lowest noise (2.25) of the three; CLAHE helps night	No technique helps sun glare
2 Segmentation	Adaptive HSV, Hough fallback	Dice 0.56, IoU 0.55, 1.8 ms; Hough rescues circular signs	Only 67 % of regions correct; fails on blur, fading, branches
3 Boundary / region	Canny; HSV contour	Best F1 (0.37), 1.4 ms; best mean IoU (0.55)	Shape classifier is weak
4 Defect check	Colour statistics + tree (depth 8)	Accuracy 0.93; false alarms 2 %; faded recall 1.00	Damaged recall 0.60; needs exact box
5 Normalisation	Box crop 64 × 64 (C if tilt ≥ 10° is known)	Oracle accuracy 0.99; 0.06 ms; C better only under tilt	Not tilt-robust
6 Recognition	HOG + Lab grid + RBF SVM	92 % on signs found	Trained on 135 generated scenes

5.8 Final Decision for Task 5

For the recognition-ready output I select **A. the plain bounding-box crop resized to 64 × 64** as the default normalisation. Evidence: it has the highest accuracy with the oracle region (0.99) and

in the full pipeline (0.63), the best accuracy when the sign is found (92 %), and it costs only 0.06 ms. I do not select C, although it is the most robust to tilt (0.85 against 0.65 at +16°), because it is 65 times slower, depends on a good mask (it was the worst technique in the full pipeline, 0.54) and loses on upright signs. I recommend it as an option for installations where a camera tilt of 10° or more is known; I did not test an automatic switch between A and C, so this is a design suggestion and not a measured result. B is rejected: it has the worst tilt behaviour (0.66 at +16° and 0.54 at −16°).

For the pipeline as a whole I defend the order enhance → segment → boundary → defect check → normalise → recognise, with the following reasoning. Enhancement comes first because the segmentation of Task 2 was measured on enhanced frames. The boundary stage runs before the defect check so that the shape is known. Normalisation comes last because it needs the final mask and box, and the recogniser needs the normalised crop. The total time of 23.3 ms leaves about 10 ms of a 33-ms frame for capture, tracking and display. **Limits:** the weakest stage is detection (67 % of the signs found, only 0.27 under motion blur), so the overall recognition accuracy is 0.63 and not the 0.99 of the oracle test; all data are generated; and no deep-learning detector (YOLO, Faster R-CNN) or recogniser was run, so I make no claim about them. The first improvement that I would test is the Wiener deblurring of Task 1 before segmentation, and a relaxed saturation threshold for faded signs.

6. Overall Conclusion, Limitations and References

6.1 Summary of the Selected Techniques

Table 38: Techniques selected for the five tasks and the key evidence

Task	Selected technique	Key evidence (test scenes)	Main limit
1 Enhancement	Gamma on L (CLAHE for night)	Dice 0.55, noise 2.25; Wiener for blur, median for rain	No gain on sun frames
2 Segmentation	Adaptive HSV threshold	Dice 0.56, IoU 0.55, 58 % detected, 1.8 ms	Target Dice 0.80 met only for normal, shadow, rain
3 Boundary / region	Canny + HSV contour	F1 0.37; IoU 0.55; 1.4 ms and 1.9 ms	Weak shape classification
4 Defect detection	Ring colour statistics + tree	Accuracy 0.93; false alarms 2 %	Damaged recall 0.60; box must be exact
5 Output and pipeline	Box crop 64 × 64; HOG + SVM	92 % on found signs; 23.3 ms per frame	Only 67 % of signs found

Taken together, the system meets the real-time target (23.3 ms against 33 ms) and gives readable output for signs that are found, but it does not yet meet the accuracy target on every condition. Colour is the strongest cue in every task: the red of a sign carries the segmentation, the defect check and the colour of the normalised output. It is also the weakest point: whenever the colour is lost (night, motion blur, fading, glare) the pipeline loses the sign. This is the reason why the next version of the system should add a shape-based detector that does not depend on colour.

6.2 Assumptions Made in This Report

- Images are 640 × 480 colour road scenes; the sign is 84–132 px wide (Table 1).
- Variants 0–2 are used only to choose thresholds, tree depths and to train the recognisers; variants 3–5 (135 scenes) are used for every reported number.
- In Task 4 the sign box is assumed to be known; Section 4.5 and Section 5.6 measure what happens when it is not.
- In Task 4 the six conditions that are not defects (normal, sun, night, rain, shadow, blur) are labelled ok.
- In Task 5 a frame with no detected sign counts as a recognition error; the success criteria of Tasks 4 and 5 were set by me because the scenario gives no numbers.

6.3 Limitations and Honest Statement

All images are generated, so real reflections, lens flare, wet roads, dirt, perspective and the exact look of real signs are missing; no accuracy claim is made for real roads. Deep-learning detectors

and recognisers (YOLO, Faster R-CNN, U-Net, CNN classifiers trained on GTSRB) were not run because no GPU, model download or dataset was available; they are mentioned only as the natural next step. The test set has 15 scenes per condition, so a single scene changes a per-condition score by 0.07 and differences of 0.05 between techniques are within the noise of this set.

The scene generator was re-created for Tasks 4 and 5 (Appendix A). It follows exactly the specification of Table 2 (five signs, nine conditions, three calibration and three test variants, fixed seeds), but it is not the same program file that produced the numbers of Tasks 1–3, so the scenes of Tasks 4–5 are statistically similar to those of Tasks 1–3 and not pixel-identical. The Task 4–5 results can be reproduced exactly by running the code of Appendix A.

6.4 Viva Preparation

- Run the final pipeline on a few real road photographs (for example from GTSRB) and report how many signs are found.
- Explain why normalisation A beats C on upright signs but not under tilt.
- Explain why a displacement of 3 px lowers the accuracy of the defect check from 0.93 to 0.77.
- Explain one deep-learning alternative (YOLO for detection) and what it would change in the pipeline.

6.5 References

[1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[2] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.

[3] N. Otsu, “A threshold selection method from gray-level histograms,” IEEE Trans. Systems, Man and Cybernetics, 9(1), 1979.

[4] J. Canny, “A computational approach to edge detection,” IEEE Trans. PAMI, 8(6), 1986.

[5] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in Graphics Gems IV, Academic Press, 1994.

[6] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” CVPR, 2005.

[7] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, 20(3), 1995.

[8] L. Breiman, J. Friedman, R. Olshen and C. Stone, Classification and Regression Trees, Wadsworth, 1984.

[9] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "Man vs. computer: benchmarking machine learning algorithms for traffic sign recognition," Neural Networks, 32, 2012.

[10] N. Wiener, Extrapolation, Interpolation and Smoothing of Stationary Time Series, MIT Press, 1949.

Appendix A – Complete Source Code for Tasks 4 and 5

The listings below are the complete, commented programs used for Tasks 4 and 5. They are run in the order scene.py → cvlib.py → task4.py → task5.py with Python 3, OpenCV 5.0, NumPy, SciPy and scikit-learn. Every scene is created from a fixed seed, so running them again gives the same scenes and the same numbers.

A.1 scene.py – scene generator with exact ground truth

```
"""scene.py - synthetic road scenes with exact ground truth (640 x 480, BGR)."""
import zlib
import cv2
import numpy as np


W, H = 640, 480
SIGNS = ['stop', 'speed30', 'speed60', 'noentry', 'pedestrian']
CONDS = ['normal', 'sun', 'night', 'rain', 'shadow', 'blur', 'faded', 'hidden', 'damaged']
RED = (35, 30, 200)  # BGR



def _sprite(kind, s):
    """Draw a sign of size s x s; returns BGR sprite and its shape mask (S x S)."""
    S = s * 3                                   # 3x supersampling for clean edges
    img = np.full((S, S, 3), 255, np.uint8); m = np.zeros((S, S), np.uint8); c = S // 2; r = int(S * 0.48)
    if kind == 'stop':
        pts = np.array([[c + r * np.cos(np.pi / 8 + k * np.pi / 4), c + r * np.sin(np.pi / 8 + k * np.pi /
4)] for k in range(8)], np.int32)
        cv2.fillPoly(m, [pts], 255); cv2.fillPoly(img, [pts], RED)
        pts2 = (c + (pts - c) * 0.9).astype(np.int32); cv2.polylines(img, [pts2], True, (255, 255, 255),
max(2, S // 60))
        cv2.putText(img, 'STOP', (int(S * 0.17), int(S * 0.6)), cv2.FONT_HERSHEY_SIMPLEX, S / 120, (255,
255, 255), max(3, S // 30))
    elif kind in ('speed30', 'speed60'):
        cv2.circle(m, (c, c), r, 255, -1); cv2.circle(img, (c, c), r, RED, -1); cv2.circle(img, (c, c),
int(r * 0.72), (255, 255, 255), -1)
        cv2.putText(img, kind[-2:], (int(S * 0.2), int(S * 0.64)), cv2.FONT_HERSHEY_DUPLEX, S / 100, (0, 0,
0), max(3, S // 28))
    elif kind == 'noentry':
        cv2.circle(m, (c, c), r, 255, -1); cv2.circle(img, (c, c), r, RED, -1)
        cv2.rectangle(img, (int(S * 0.2), int(S * 0.42)), (int(S * 0.8), int(S * 0.58)), (255, 255, 255), -
1)
    else:
        pts = np.array([[c, int(S * 0.05)], [int(S * 0.97), int(S * 0.92)], [int(S * 0.03), int(S * 0.92)]],
np.int32)
        cv2.fillPoly(m, [pts], 255); cv2.fillPoly(img, [pts], RED)
        inner = (np.array([c, int(S * 0.58)]) + (pts - np.array([c, int(S * 0.58)])) *
0.68).astype(np.int32)
        cv2.fillPoly(img, [inner], (255, 255, 255))
        cv2.circle(img, (c, int(S * 0.5)), int(S * 0.045), (0, 0, 0), -1)
        cv2.line(img, (c, int(S * 0.55)), (c, int(S * 0.7)), (0, 0, 0), max(3, S // 40))
        cv2.line(img, (c, int(S * 0.7)), (c - int(S * 0.06), int(S * 0.82)), (0, 0, 0), max(3, S // 40))
        cv2.line(img, (c, int(S * 0.7)), (c + int(S * 0.06), int(S * 0.82)), (0, 0, 0), max(3, S // 40))
    img = cv2.resize(img, (s, s), interpolation=cv2.INTER_AREA); m = cv2.resize(m, (s, s),
interpolation=cv2.INTER_AREA)
    return img, m



def clean_template(kind, size=64):
    img, m = _sprite(kind, size)
    return img, (m > 127)



def make_scene(kind, cond, variant):
    seed = zlib.crc32(f'{kind}-{cond}-{variant}'.encode()) & 0xffffffff
    rng = np.random.RandomState(seed)
    img = np.zeros((H, W, 3), np.float32)
    top = np.array([235, 200, 150], np.float32); bot = np.array([250, 235, 215], np.float32)      # sky
(BGR)
    for y in range(H // 2 + 40):
        img[y] = top + (bot - top) * y / (H // 2 + 40)
```

```
    img[H // 2 + 40:] = (110, 110, 112)                                                          # road
    for _ in range(7):                                                                           #
buildings
        x = rng.randint(-20, W - 40); w = rng.randint(70, 150); h = rng.randint(80, 190)
        col = [(60, 70, 160), (70, 85, 175), (170, 175, 180), (150, 160, 165)][rng.randint(4)]     # some red
brick
        cv2.rectangle(img, (x, H // 2 + 40 - h), (x + w, H // 2 + 40), col, -1)
        for wy in range(H // 2 + 50 - h, H // 2 + 30, 26):
            for wx in range(x + 8, x + w - 14, 24):
                cv2.rectangle(img, (wx, wy), (wx + 12, wy + 14), (190, 170, 130), -1)
    for _ in range(6):                                                                           # trees
        x = rng.randint(0, W); y = H // 2 + 20 - rng.randint(0, 20)
        cv2.rectangle(img, (x - 4, y), (x + 4, y + 50), (40, 70, 110), -1)
        cv2.ellipse(img, (x, y - 10), (rng.randint(30, 50), rng.randint(30, 45)), 0, 0, 360, (50, 140 +
rng.randint(30), 40), -1)
    cv2.fillPoly(img, [np.array([[W // 2 - 40, H // 2 + 40], [W // 2 + 40, H // 2 + 40], [W - 20, H], [20,
H]], np.int32)], (95, 95, 98))
    vx = rng.randint(60, 300); cv2.rectangle(img, (vx, H // 2 + 60), (vx + 90, H // 2 + 110), (30, 30, 190),
-1)   # red vehicle
    cv2.rectangle(img, (vx + 15, H // 2 + 45), (vx + 70, H // 2 + 62), (40, 40, 170), -1)
    s = rng.randint(84, 133); sx = rng.randint(40, W - s - 40); sy = rng.randint(30, 160)
    cv2.rectangle(img, (sx + s // 2 - 3, sy + s - 4), (sx + s // 2 + 3, H // 2 + 60), (120, 120, 120), -1)
# pole
    spr, sm = _sprite(kind, s)
    full = np.zeros((H, W), bool); full[sy:sy + s, sx:sx + s] = sm > 127
    a = np.zeros((H, W), np.float32); a[sy:sy + s, sx:sx + s] = sm / 255.0
    layer = np.zeros_like(img); layer[sy:sy + s, sx:sx + s] = spr
    damage = np.zeros((H, W), bool); branch = np.zeros((H, W), bool)
    sub = (slice(sy, sy + s), slice(sx, sx + s))
    if cond == 'faded':                                          # 58 % blend towards grey, inside the sign only
        layer = layer * 0.42 + 150 * 0.58
    if cond == 'damaged':                                        # peeled patches (pale) and scratches
        dm = np.zeros((s, s), np.uint8)
        for _ in range(rng.randint(3, 6)):
            cv2.circle(dm, (rng.randint(8, s - 8), rng.randint(8, s - 8)), rng.randint(5, 11), 255, -1)
        for _ in range(rng.randint(2, 5)):
            p = rng.randint(5, s - 5, 2); q = rng.randint(5, s - 5, 2); cv2.line(dm, tuple(int(v) for v in
p), tuple(int(v) for v in q), 255, 2)
        d = (dm > 0) & (sm > 127); layer[sub][d] = (175, 178, 182); damage[sub] = d
    img = img * (1 - a[..., None]) + layer * a[..., None]
    if cond == 'hidden':                                         # green branches in front of the sign
        bm = np.zeros((H, W), np.uint8); side = rng.randint(2)
        for k in range(rng.randint(3, 6)):
            x0 = sx - 10 if side == 0 else sx + s + 10; y0 = sy + rng.randint(0, s)
            x1 = sx + rng.randint(s // 4, s // 2) if side == 0 else sx + s - rng.randint(s // 4, s // 2); y1
= y0 + rng.randint(-25, 25)
            cv2.line(bm, (x0, y0), (x1, y1), 255, rng.randint(7, 12))
            cv2.ellipse(bm, (x1, y1), (rng.randint(10, 18), rng.randint(8, 14)), rng.randint(0, 180), 0,
360, 255, -1)
        branch = bm > 0
        img[branch] = (45, 120 + rng.randint(0, 30), 45)
    visible = full & ~branch
    sign_c = (sx + s // 2, sy + s // 2)
    if cond == 'sun':
        g = img / 255.0; img = 255 * g ** 0.6 * 1.05
        yy, xx = np.mgrid[0:H, 0:W]; glare = np.exp(-((xx - sign_c[0] + 40) ** 2 + (yy - sign_c[1] + 40) **
2) / (2 * 70.0 ** 2))
        img = img + 70 * glare[..., None]; gray = img.mean(2, keepdims=True); img = gray + (img - gray) *
0.6
    elif cond == 'night':
        img = img * 0.16 + np.array([10, 4, 0])
        if variant % 2 == 0:
            yy, xx = np.mgrid[0:H, 0:W]; img += 55 * np.exp(-((xx - sign_c[0]) ** 2 + (yy - sign_c[1] - 120)
** 2) / (2 * 80.0 ** 2))[..., None]
        img = img + rng.normal(0, 7, img.shape)
    elif cond == 'rain':
        img = img * 0.8 + 0.2 * 200; rl = np.zeros((H, W), np.uint8)
        for _ in range(250):
            x = rng.randint(0, W); y = rng.randint(0, H); cv2.line(rl, (x, y), (x - 4, y + 18), 255, 1)
        img = img + 0.5 * cv2.GaussianBlur(rl, (3, 3), 0)[..., None].astype(np.float32); img =
cv2.GaussianBlur(img, (0, 0), 1.0)
    elif cond == 'shadow':
        yy, xx = np.mgrid[0:H, 0:W]; x0 = sign_c[0] + rng.randint(-30, 30)
        sh = 1 / (1 + np.exp((xx - x0) / 12.0)); img = img * (0.45 + 0.55 * (1 - sh))[..., None]
    elif cond == 'blur':
        L = rng.randint(15, 26); k = np.zeros((L, L), np.float32); k[L // 2] = 1.0 / L; img =
cv2.filter2D(img, -1, k)
    img = np.clip(img + rng.normal(0, 1.5, img.shape), 0, 255).astype(np.uint8)
    ys, xs = np.where(full)
    return dict(img=img, full=full, visible=visible, branch=branch & full, damage=damage,
bbox=(int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1),
                kind=kind, cond=cond, variant=variant)
```

```
def boundary_of(mask):
    m = mask.astype(np.uint8); return (m - cv2.erode(m, np.ones((3, 3), np.uint8))) > 0
```

A.2 cvlib.py – metrics, enhancement, segmentation and edge helpers

```
"""cvlib.py - shared helpers (metrics, Task 1 enhancement, Task 2 segmentation, Task 3 edges)."""
import time
import cv2
import numpy as np
from scene import W, H

K5 = np.ones((5, 5), np.uint8)



def dice(p, t):
    p = p.astype(bool); t = t.astype(bool); return 2 * (p & t).sum() / max(p.sum() + t.sum(), 1)



def iou(p, t):
    p = p.astype(bool); t = t.astype(bool); return (p & t).sum() / max((p | t).sum(), 1)



def tmean(fn, *a, n=15):
    fn(*a); t0 = time.perf_counter()
    for _ in range(n): fn(*a)
    return 1000 * (time.perf_counter() - t0) / n



def _on_L(bgr, f):
    lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB); lab[..., 0] = f(lab[..., 0]); return cv2.cvtColor(lab,
cv2.COLOR_LAB2BGR)



def enh_gamma(bgr):
    def f(L):
        m = np.clip(L.mean() / 255.0, 0.02, 0.98); g = np.clip(np.log(0.5) / np.log(m), 0.35, 3.0)
        return cv2.LUT(L, (255 * (np.arange(256) / 255.0) ** g).astype(np.uint8))
    return _on_L(bgr, f)



def enh_clahe(bgr, clip=2.0, tile=8):
    cl = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tile, tile)); return _on_L(bgr, cl.apply)



def enhance(bgr):
    """Task 1 decision: gamma by default, CLAHE for very dark frames (median L < 60)."""
    L = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)[..., 0]
    return enh_clahe(bgr) if np.median(L) < 60 else enh_gamma(bgr)



def fill_holes(m):
    cs, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); out = np.zeros_like(m);
cv2.drawContours(out, cs, -1, 255, -1); return out



def pick_sign(mask):
    cs, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE); best, bs = None, 0
    for c in cs:
        a = cv2.contourArea(c); x, y, w, h = cv2.boundingRect(c)
        if not (400 <= a <= 16000) or min(w, h) / max(w, h) < 0.75: continue
        hull = cv2.contourArea(cv2.convexHull(c)); sol = a / max(hull, 1)
        if sol < 0.8: continue
        s = a * sol * (min(w, h) / max(w, h))
        if s > bs: best, bs = c, s
    out = np.zeros(mask.shape, np.uint8)
    if best is not None: cv2.drawContours(out, [best], -1, 255, -1)
    return out > 0
```

```
def seg_hsv(bgr, ratio=0.8, v_min=45):
    h, s, v = cv2.split(cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV))
    redhue = ((h <= 10) | (h >= 170)) & (v >= v_min) & (s >= 40)
    if redhue.sum() < 200: return np.zeros(s.shape, bool)
    thr = max(40, ratio * np.percentile(s[redhue], 98))
    m = ((redhue & (s >= thr)).astype(np.uint8)) * 255
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K5); m = cv2.morphologyEx(m, cv2.MORPH_OPEN, np.ones((3, 3),
np.uint8))
    return pick_sign(fill_holes(m))



def roi_hough(bgr):
    g = cv2.medianBlur(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY), 5)
    c = cv2.HoughCircles(g, cv2.HOUGH_GRADIENT, 1, 50, param1=120, param2=28, minRadius=18, maxRadius=75)
    m = np.zeros(g.shape, np.uint8)
    if c is not None: x, y, r = c[0][0]; cv2.circle(m, (int(x), int(y)), int(r), 255, -1)
    return m > 0



def edge_canny(bgr):
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY); g = cv2.GaussianBlur(g, (0, 0), 1.4)
    hi, _ = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU); hi = 0.5 * hi
    return cv2.Canny(g, 0.5 * hi, hi) > 0



def bbox_of(mask):
    ys, xs = np.where(mask)
    if len(xs) == 0: return None
    return int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1
```

A.3 task4.py – defect detection with three techniques

```
"""task4.py - detection of faded, partly hidden and damaged signs (three techniques)."""
import json, pickle, time
import cv2
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from scene import *
from cvlib import *

S = 64
DEFECT = {'faded': 'faded', 'hidden': 'hidden', 'damaged': 'damaged'}
CLASSES = ['ok', 'faded', 'hidden', 'damaged']



def label_of(cond): return DEFECT.get(cond, 'ok')



def _tmpl(kind):
    from scene import _sprite
    spr, m = _sprite(kind, 128); ys, xs = np.where(m > 127)
    sl = (slice(ys.min(), ys.max() + 1), slice(xs.min(), xs.max() + 1))
    return cv2.resize(spr[sl], (S, S), interpolation=cv2.INTER_AREA), cv2.resize((m[sl] >
127).astype(np.uint8), (S, S), interpolation=cv2.INTER_NEAREST) > 0



TEMPL = {k: _tmpl(k) for k in SIGNS}
TG = {k: cv2.cvtColor(v[0], cv2.COLOR_BGR2GRAY).astype(np.float32) for k, v in TEMPL.items()}



def crop_of(img, bbox, jit=(0, 0)):
    x0, y0, x1, y1 = bbox; dx, dy = jit
    x0, x1 = max(x0 + dx, 0), min(x1 + dx, W); y0, y1 = max(y0 + dy, 0), min(y1 + dy, H)
    return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)



def best_template(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY).astype(np.float32); best = ('', -2.0)
```

```
    for k, t in TG.items():
        m = TEMPL[k][1]
        a = g[m] - g[m].mean(); b = t[m] - t[m].mean(); r = (a * b).sum() / (np.linalg.norm(a) *
np.linalg.norm(b) + 1e-6)
        if r > best[1]: best = (k, float(r))
    return best



# ---------------------- A. colour-statistics (fading) ----------------------
def tech_A(crop):
    k, _ = best_template(crop); m = TEMPL[k][1]
    ring = m & ~(cv2.erode(m.astype(np.uint8), np.ones((3, 3), np.uint8), iterations=9) > 0)
    h, s, v = cv2.split(cv2.cvtColor(crop, cv2.COLOR_BGR2HSV))
    red = ((h <= 10) | (h >= 170)) & (s >= 90) & (v >= 60)
    notred = ring & ~red
    feats = [s[ring].mean(), red[ring].mean(), v[ring].std(), s[m].mean(), notred.sum() / max(ring.sum(),
1)]
    return np.array(feats, np.float32), notred



# ---------------------- B. template comparison ----------------------
def tech_B(crop):
    k, ncc = best_template(crop); m = TEMPL[k][1]
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY).astype(np.float32); t = TG[k]
    gn = (g - g[m].mean()) / (g[m].std() + 1e-6); tn = (t - t[m].mean()) / (t[m].std() + 1e-6)
    d = np.abs(gn - tn) * m
    mp = (d > 1.3).astype(np.uint8); mp = cv2.morphologyEx(mp, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
    n, lbl, st, _ = cv2.connectedComponentsWithStats(mp)
    big = st[1:, cv2.CC_STAT_AREA].max() if n > 1 else 0
    colour = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB).astype(np.float32); tc = cv2.cvtColor(TEMPL[k][0],
cv2.COLOR_BGR2LAB).astype(np.float32)
    dc = np.linalg.norm(colour - tc, axis=2)[m].mean()
    feats = [ncc, mp.sum() / m.sum(), big / m.sum(), d[m].mean(), dc]
    return np.array(feats, np.float32), mp > 0



# ---------------------- C. shape completeness ----------------------
def tech_C(crop):
    h, s, v = cv2.split(cv2.cvtColor(crop, cv2.COLOR_BGR2HSV))
    red = ((h <= 12) | (h >= 168)) & (s >= 50) & (v >= 40)
    m = red.astype(np.uint8) * 255; m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((3, 3), np.uint8))
    f = fill_holes(m); cs, _ = cv2.findContours(f, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not cs: return np.array([0, 0, 0, 0, 0, 0], np.float32), np.zeros((S, S), bool)
    c = max(cs, key=cv2.contourArea); a = cv2.contourArea(c); hull = cv2.convexHull(c); ha =
cv2.contourArea(hull)
    hm = np.zeros((S, S), np.uint8); cv2.drawContours(hm, [hull], -1, 255, -1)
    conc = (hm > 0) & (f == 0)
    depth = 0.0
    hi = cv2.convexHull(c, returnPoints=False)
    if hi is not None and len(hi) > 3:
        try:
            df = cv2.convexityDefects(c, hi); depth = 0.0 if df is None else df.reshape(-1, 4)[:, 3].max() /
256.0 / S
        except cv2.error: depth = 0.0
    ncomp = cv2.connectedComponents(m)[0] - 1
    feats = [a / (S * S), a / max(ha, 1), depth, conc.sum() / max(ha, 1), ncomp, (f > 0).sum() / (S * S)]
    return np.array(feats, np.float32), conc



TECH = {'A': tech_A, 'B': tech_B, 'C': tech_C}



def all_scenes(variants):
    return [make_scene(k, c, v) for v in variants for k in SIGNS for c in CONDS]



def prep(scene):
    return enhance(scene['img'])



def run_features(scenes, jit=(0, 0)):
    out = []
    for sc in scenes:
        img = prep(sc); crop = crop_of(img, sc['bbox'], jit); rec = {}
        for n, f in TECH.items(): rec[n] = f(crop)
        out.append(rec)
```

```
    return out



def gt_map(sc):
    x0, y0, x1, y1 = sc['bbox']; g = (sc['branch'] | sc['damage']) & sc['full']
    return cv2.resize(g[y0:y1, x0:x1].astype(np.uint8), (S, S), interpolation=cv2.INTER_NEAREST) > 0



if __name__== '__main__':
    cal = all_scenes([0, 1, 2]); tst = all_scenes([3, 4, 5])
    fc = run_features(cal); ft = run_features(tst)
    yc = np.array([CLASSES.index(label_of(s['cond'])) for s in cal]); yt =
np.array([CLASSES.index(label_of(s['cond'])) for s in tst])
    R = {}
    models = {}
    from sklearn.model_selection import cross_val_score, StratifiedKFold
    cvd = {}; depth = {}
    for n in 'ABC':
        Xc = np.stack([r[n][0] for r in fc]); Xt = np.stack([r[n][0] for r in ft])
        cvd[n] = {d: float(cross_val_score(DecisionTreeClassifier(max_depth=d, class_weight='balanced',
random_state=0), Xc, yc, cv=StratifiedKFold(5, shuffle=True, random_state=0)).mean()) for d in (2, 3, 4, 5,
6, 8)}
        depth[n] = max(cvd[n], key=lambda d: (round(cvd[n][d], 3), -d))
        clf = DecisionTreeClassifier(max_depth=depth[n], class_weight='balanced', random_state=0).fit(Xc,
yc); models[n] = clf
        p = clf.predict(Xt); R[n] = dict(pred=p.tolist())
    Xc = np.hstack([np.stack([r[n][0] for r in fc]) for n in 'ABC']); Xt = np.hstack([np.stack([r[n][0] for
r in ft]) for n in 'ABC'])
    clf = DecisionTreeClassifier(max_depth=6, class_weight='balanced', random_state=0).fit(Xc, yc); R['F'] =
dict(pred=clf.predict(Xt).tolist())
    # sweeps: depth and jitter for each technique
    sw = {}
    for n in 'ABC':
        Xc = np.stack([r[n][0] for r in fc]); Xt = np.stack([r[n][0] for r in ft])
        sw[n] = {d: float((DecisionTreeClassifier(max_depth=d, class_weight='balanced',
random_state=0).fit(Xc, yc).predict(Xt) == yt).mean()) for d in (1, 2, 3, 4, 6, 8)}
    jit_res = {}
    for j in (0, 3, 6, 10):
        ftj = run_features(tst, (j, j // 2)); jit_res[j] = {}
        for n in 'ABC':
            Xtj = np.stack([r[n][0] for r in ftj]); jit_res[j][n] = float((models[n].predict(Xtj) ==
yt).mean())
    # localisation
    loc = {n: {'hidden': [], 'damaged': []} for n in 'ABC'}; occ = {n: [] for n in 'ABC'}
    for sc, r in zip(tst, ft):
        if sc['cond'] in ('hidden', 'damaged'):
            g = gt_map(sc)
            for n in 'ABC': loc[n][sc['cond']].append(dice(r[n][1], g))
            if sc['cond'] == 'hidden':
                a = TEMPL[sc['kind']][1].sum()
                for n in 'ABC': occ[n].append(abs(r[n][1].sum() / a - g.sum() / a))
    # timing on one crop
    sc0 = tst[0]; crop0 = crop_of(prep(sc0), sc0['bbox']); tm = {n: tmean(TECH[n], crop0, n=40) for n in
'ABC'}
    pickle.dump(dict(cvd=cvd, depth=depth, R=R, yt=yt, yc=yc, sw=sw, jit=jit_res, loc=loc, occ=occ, tm=tm,
cal_n=len(cal), tst=[(s['kind'], s['cond'], s['variant']) for s in tst],
                     Xt={n: np.stack([r[n][0] for r in ft]) for n in 'ABC'}, Xc={n: np.stack([r[n][0] for r
in fc]) for n in 'ABC'}), open('/tmp/w/t4.pkl', 'wb'))
    import sys
    for n in 'ABCF':
        p = np.array(R[n]['pred']); print(n, 'acc', (p == yt).mean(), 'ok-false-alarm', (p[yt == 0] !=
0).mean(), [float((p[yt == i] == i).mean()) for i in (1, 2, 3)])
    print(cvd, depth);print(sw); print(jit_res); print({n: {k: float(np.mean(v)) for k, v in loc[n].items()}
for n in loc}); print({n: float(np.mean(occ[n])) for n in occ}); print(tm)
```

A.4 task5.py – normalisation, recognition and pipeline

```
"""task5.py - recognition-ready output (three normalisations), recognition accuracy, integrated pipeline."""
import pickle, time
import cv2
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from scene import *
from cvlib import *
import task4 as T4
```

```
S = 64
from skimage.feature import hog as _hog



# ---------------- three normalisation techniques ----------------
def norm_A(img, bbox, mask):
    """A. Plain bounding-box crop resized to 64 x 64."""
    x0, y0, x1, y1 = bbox; return cv2.resize(img[y0:y1, x0:x1], (S, S), interpolation=cv2.INTER_AREA)



def _margin_box(bbox, mask, m=0.08):
    x0, y0, x1, y1 = bbox; pw = int((x1 - x0) * m); ph = int((y1 - y0) * m)
    return max(x0 - pw, 0), max(y0 - ph, 0), min(x1 + pw, W), min(y1 + ph, H)



def norm_B(img, bbox, mask):
    """B. Mask-guided crop: background outside the sign replaced by neutral grey, colour/brightness
standardised inside the mask."""
    x0, y0, x1, y1 = _margin_box(bbox, mask); im = img[y0:y1, x0:x1].astype(np.float32); m = mask[y0:y1,
x0:x1]
    if m.sum() < 20: return cv2.resize(img[bbox[1]:bbox[3], bbox[0]:bbox[2]], (S, S),
interpolation=cv2.INTER_AREA)
    out = np.full_like(im, 128.0)
    for c in range(3):
        v = im[..., c]; mu, sd = v[m].mean(), v[m].std() + 1e-3
        out[..., c][m] = np.clip(128 + (v[m] - mu) * 55.0 / sd * 0.6 + (mu - 128) * 0.0, 0, 255)
    return cv2.resize(out.astype(np.uint8), (S, S), interpolation=cv2.INTER_AREA)



def norm_C(img, bbox, mask):
    """C. Contour-aligned: de-skew with the minimum-area rectangle, grey background, CLAHE on L, aspect-
preserving pad to 64 x 64."""
    m8 = mask.astype(np.uint8); cs, _ = cv2.findContours(m8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not cs: return norm_A(img, bbox, mask)
    c = max(cs, key=cv2.contourArea); (cx, cy), (rw, rh), ang = cv2.minAreaRect(c)
    ang = ang % 90.0; ang = ang - 90.0 if ang > 45 else ang                # smallest rotation that aligns
the box
    M = cv2.getRotationMatrix2D((cx, cy), ang, 1.0)
    ri = cv2.warpAffine(img, M, (W, H), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE); rm =
cv2.warpAffine(m8, M, (W, H), flags=cv2.INTER_NEAREST) > 0
    bb = bbox_of(rm)
    if bb is None: return norm_A(img, bbox, mask)
    x0, y0, x1, y1 = _margin_box(bb, rm, 0.05); im = ri[y0:y1, x0:x1].copy(); m = rm[y0:y1, x0:x1]
    lab = cv2.cvtColor(im, cv2.COLOR_BGR2LAB); lab[..., 0] = cv2.createCLAHE(2.0, (4, 4)).apply(lab[...,
0]); im = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
    im[~m] = 128; h, w = im.shape[:2]; sc = S / max(h, w)
    im = cv2.resize(im, (max(1, int(round(w * sc))), max(1, int(round(h * sc)))),
interpolation=cv2.INTER_AREA)
    out = np.full((S, S, 3), 128, np.uint8); oy = (S - im.shape[0]) // 2; ox = (S - im.shape[1]) // 2
    out[oy:oy + im.shape[0], ox:ox + im.shape[1]] = im; return out



NORM = {'A': norm_A, 'B': norm_B, 'C': norm_C}



def feat(crop):
    g = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY); h = _hog(g, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2, 2)).ravel()
    lab = cv2.cvtColor(crop, cv2.COLOR_BGR2LAB).astype(np.float32); grid = cv2.resize(lab, (4, 4),
interpolation=cv2.INTER_AREA).ravel() / 255.0
    return np.concatenate([h, grid]).astype(np.float32)



# ---------------- detection stage (Tasks 1-3 decisions) ----------------
def detect(img):
    e = enhance(img); m = seg_hsv(e); how = 'hsv'
    if not m.any(): m = seg_hsv(img); how = 'hsv-raw'
    if not m.any(): m = roi_hough(e); how = 'hough'
    if not m.any(): return None, None, 'none'
    return bbox_of(m), m, how



def rotate_scene(sc, ang):
```

```
    cx = (sc['bbox'][0] + sc['bbox'][2]) / 2; cy = (sc['bbox'][1] + sc['bbox'][3]) / 2; M =
cv2.getRotationMatrix2D((cx, cy), ang, 1.0)
    img = cv2.warpAffine(sc['img'], M, (W, H), borderMode=cv2.BORDER_REPLICATE); full =
cv2.warpAffine(sc['full'].astype(np.uint8), M, (W, H), flags=cv2.INTER_NEAREST) > 0
    return img, full



if __name__== '__main__':
    cal = T4.all_scenes([0, 1, 2]); tst = T4.all_scenes([3, 4, 5])
    ycal = np.array([SIGNS.index(s['kind']) for s in cal]); ytst = np.array([SIGNS.index(s['kind']) for s in
tst])
    # train one SVM per normalisation on calibration crops taken from the true box and mask
    clfs = {}
    for n, f in NORM.items():
        X = np.stack([feat(f(s['img'], s['bbox'], s['full'])) for s in cal])
        clfs[n] = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=10, gamma='scale')).fit(X, ycal)
    # (i) oracle region: isolates the effect of the normalisation
    orc = {}
    for n, f in NORM.items():
        p = np.array([clfs[n].predict([feat(f(s['img'], s['bbox'], s['full']))])[0] for s in tst]); orc[n] =
p
    # (ii) full pipeline: detector output feeds the normalisation
    det = [detect(s['img']) for s in tst]
    ok_det = np.array([d[0] is not None and iou(d[1], s['full']) >= 0.5 for d, s in zip(det, tst)])
    pipe = {}
    for n, f in NORM.items():
        p = []
        for d, s in zip(det, tst):
            if d[0] is None: p.append(-1); continue
            p.append(clfs[n].predict([feat(f(s['img'], d[0], d[1]))])[0])
        pipe[n] = np.array(p)
    # (iii) tilt what-if (oracle region on rotated scenes)
    tilt = {}
    for ang in (0, 8, 16, -16):
        tilt[ang] = {}
        for n, f in NORM.items():
            p = []
            for s in tst:
                img, full = rotate_scene(s, ang) if ang else (s['img'], s['full']); bb = bbox_of(full)
                p.append(clfs[n].predict([feat(f(img, bb, full))])[0])
            tilt[ang][n] = float((np.array(p) == ytst).mean())
    # stage timing (mean over 135 test scenes) for the final pipeline: gamma/CLAHE -> HSV -> Canny -> defect
check -> C normalisation -> SVM
    A_tree = None
    fc = T4.run_features(cal); Xa = np.stack([r['A'][0] for r in fc]); yc =
np.array([T4.CLASSES.index(T4.label_of(s['cond'])) for s in cal])
    A_tree = DecisionTreeClassifier(max_depth=8, class_weight='balanced', random_state=0).fit(Xa, yc)
    st = dict(enh=[], seg=[], edge=[], defect=[], norm=[], rec=[]); defect_pred = []
    for s in tst:
        t0 = time.perf_counter(); e = enhance(s['img']); t1 = time.perf_counter()
        bb, m, how = detect(s['img']); t2 = time.perf_counter()
        edge_canny(e); t3 = time.perf_counter()
        if bb is None: continue
        crop = T4.crop_of(e, bb); fa, _ = T4.tech_A(crop); dp = A_tree.predict([fa])[0];
defect_pred.append((T4.label_of(s['cond']), T4.CLASSES[dp])); t4 = time.perf_counter()
        nc = norm_A(s['img'], bb, m); t5 = time.perf_counter(); clfs['A'].predict([feat(nc)]); t6 =
time.perf_counter()
        for k, a, b in (('enh', t0, t1), ('seg', t1, t2), ('edge', t2, t3), ('defect', t3, t4), ('norm', t4,
t5), ('rec', t5, t6)): st[k].append(1000 * (b - a))
    stage = {k: float(np.mean(v)) for k, v in st.items()}
    # normalisation-only timing
    s0 = tst[0]; ntime = {n: tmean(lambda: NORM[n](s0['img'], s0['bbox'], s0['full']), n=40) for n in NORM}
    pickle.dump(dict(ytst=ytst, orc={k: v.tolist() for k, v in orc.items()}, pipe={k: v.tolist() for k, v in
pipe.items()}, ok_det=ok_det.tolist(),
                     how=[d[2] for d in det], tilt=tilt, stage=stage, ntime=ntime, defect_pred=defect_pred,
                     tst=[(s['kind'], s['cond'], s['variant']) for s in tst]), open('/tmp/w/t5.pkl', 'wb'))
    for n in 'ABC':
        print(n, 'oracle acc', (orc[n] == ytst).mean(), 'pipeline acc', (pipe[n] == ytst).mean(),
'acc|detected', (pipe[n][ok_det] == ytst[ok_det]).mean())
    print('detected', ok_det.mean(), {h: det_n for h, det_n in zip(*np.unique([d[2] for d in det],
return_counts=True))})
    print(tilt); print(stage, sum(stage.values())); print(ntime)
```